

Institute of Embedded Systems (InES)

Bachelorarbeit Studiengang Elektrotechnik

Entwicklung einer Smart-Waage mit offener Schnittstelle für die Industrie 4.0

Autor:innen

Thomas Lino Rocheteau

Hauptbetreuung

Kilian Brunner

Datum

05.06.2026

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit selbstständig, oder gemeinsam mit den aufgeführten Gruppenmitgliedern, verfasst habe. Ich habe ausschliesslich die im Text oder Anhang angegebenen Quellen und Hilfsmittel (auch Internetseiten und generative KI-Tools) benutzt. Ich trage die Verantwortung für die Qualität des Textes sowie die Auswahl aller Inhalte und habe sichergestellt, dass Informationen und Argumente mit geeigneten wissenschaftlichen Quellen belegt bzw. gestützt werden. Generative KI-Tools wurden summarisch unter der Angabe des Namens und des Zwecks angegeben. Bei Verfehlungen aller Art treten die Paragraphen 39 und 40 (Unredlichkeit und Verfahren bei Unredlichkeit) der ZHAW-Rahmenprüfungsordnung sowie die Bestimmungen der Disziplarmassnahmen der Hochschulordnung in Kraft.

Erklärung zum Einsatz von KI-Tools

Ich erkläre hiermit, dass ich für die Erstellung dieser Bachelortarbeit die Unterstützung von Claude [5], einem KI-gestützten Sprachmodell von Anthropic [4], genutzt habe. Dieses Modell wurde eingesetzt, um bestimmte Textabschnitte zu verfeinern, Anregungen für Formulierungen zu erhalten und komplizierte Konzepte verständlicher darzustellen. Ausserdem wurden einzelne Codeabschnitte mittels Claude entwickelt. Alle mit Hilfe von Claude erstellten Inhalte wurden von mir sorgfältig geprüft und den akademischen Standards sowie den spezifischen Anforderungen der ZHAW angepasst. Ich bestätige, dass die vorliegende Arbeit meine eigenen Ideen und Überlegungen widerspiegelt und gemäss den ethischen Richtlinien der akademischen Integrität verfasst wurde.

Ort, Datum

Winterthur, 05.06.2026

Name

Thomas Lino Rocheteau

Unterschrift



Winterthur, 5 Jun 2026



Simple electronic signature
Signed on Skribble.com

Abstract

English

This thesis advances a previously developed ESP32-based smart scale prototype for industrial parts and bulk-goods inventory management. Two aspects are addressed in depth: measurement accuracy and energy efficiency.

For measurement accuracy, the four YZC-133 load cells are each connected to a dedicated channel of the ADS1234 24-bit ADC with a PGA gain of 128 and a data rate of 10 SPS. Individual per-sensor calibration enables corner-load compensation, which was not possible with the single summed channel of the predecessor. Measurements confirm a repeatability of ± 0.1 g, though absolute accuracy is limited by mechanical imperfections of the prototype.

For energy efficiency, three hardware measures were implemented: the load-cell supply is interrupted via MOSFET before entering deep sleep; the ESP32 is powered directly through the 3V3 pin, eliminating the on-board LDO; and the buck-converter output voltage was reduced from 3.3 V to 3.0 V, saving an additional 73.4 mJ per cycle (-17.6%). Combined with software-level optimisations including the migration from the Arduino BLE stack to NimbleBLE, the active-cycle energy consumption is reduced from 6503 mJ to 689.5 mJ, a reduction of **89.4 %**. A custom PCB was designed to consolidate the power path; a component selection error in the buck converter prevents its immediate use in the current revision.

Deutsch

Diese Bachelorarbeit baut auf einem bestehenden ESP32-basierten Smart-Waagen-Prototyp für die industrielle Inventarisierung von Kleinteilen und Schüttgut auf und vertieft zwei Schwerpunkte: Messgenauigkeit und Energieeffizienz.

Zur Verbesserung der Messgenauigkeit wird jede der vier YZC-133-Wägezellen einzeln an einen dedizierten Kanal des ADS1234 (24-Bit-ADC, PGA-Faktor 128, 10 SPS) angeschlossen. Die individuelle Kalibrierung pro Sensor ermöglicht eine Ecklastkompensation, die mit dem summierten Einzelkanalansatz der Vorgängerarbeit nicht realisierbar war. Die Messungen zeigen eine Wiederholbarkeit von $\pm 0,1$ g; die Absolutgenauigkeit ist durch mechanische Unzulänglichkeiten des Prototyps eingeschränkt.

Für die Energieoptimierung wurden drei Hardwaremassnahmen umgesetzt: Die Wägezellen werden über einen MOSFET vor dem Deep-Sleep abgeschaltet; der ESP32 wird direkt über den 3V3-Pin versorgt, womit der interne Board-LDO entfällt; die Ausgangsspannung des Buck-Converters wurde von 3,3 V auf 3,0 V abgesenkt, was eine zusätzliche Einsparung von 73,4 mJ pro Zyklus ($-17,6\%$) erzielt. In Kombination mit Softwareoptimierungen – darunter die Migration vom Arduino-BLE-Stack auf NimbleBLE – sinkt der Energieverbrauch pro Aktivzyklus von 6503 mJ auf 689,5 mJ, eine Reduktion um **89,4 %**. Ein eigens entwickeltes PCB wurde zur Konsolidierung des Strompfads ausgelegt; ein Fehler bei der Schaltreglerwahl verhindert dessen unmittelbaren Einsatz in der vorliegenden Revision.

Inhaltsverzeichnis

1	Einleitung	6
1.1	Ausgangslage	6
1.2	Zielsetzung	6
1.3	Systemabgrenzung und Randbedingungen	7
1.4	Meilensteine	7
1.5	Aufbau der Arbeit	7
2	Technische Ausgangslage	8
2.1	Design Space Exploration	8
2.1.1	MCU-Auswahl: SBC-NodeMCU-ESP32-C vs. ESP32-C3 vs. nRF52840	8
3	Low-Power Optimierung	8
3.1	Motivation	8
3.2	Strompfad Schema	8
3.2.1	Ausgangslage	9
3.2.2	Hardware Redesign	9
3.3	Strommessungen und Low-Power-Optimierung	10
3.4	Übersicht Zyklus	10
3.5	Gesamtsystem Stromverbrauch	10
3.5.1	Vergleich Energieverbrauch Aktiver Zyklus	11
3.5.2	Vergleich Leistungsverbrauch Deep-Sleep	11
3.6	Vergleich Energieverbrauch ESP32	11
3.6.1	Übersicht Stromverlauf ESP32	11
3.6.2	Vergleich ESP32 Energieverbrauch	13
3.6.3	Optimierung Energieverbrauch Software	13
3.7	Verluste Hardware	15
3.7.1	Messmethode	15
3.7.2	Buck converter	15
3.7.3	LDO	16
3.7.4	ADC	17
3.8	Gesamtverbrauch und Batterielebensdauer	17
4	Optimierung Messgenauigkeit	18
4.1	Sensorik: Wägezellen und ADC	18
4.1.1	Wägezellen (YZC-133)	18
4.1.2	ADS1234 – 4-Kanal-24-Bit-ADC	18
4.1.3	Von einem auf vier ADC-Kanäle	18
4.2	ADS1234 – Ansteuerung und Kalibrierung	19
4.2.1	Überblick	19
4.2.2	Seriellles Leseprotokoll	19
4.2.3	Kanalwechsel CH4 → CH1: !PDWN-Workaround	20
4.2.4	Kalibrierung	20
4.3	Messreihe Messgenauigkeit	21
5	Geräteübersicht	22
6	PCB-Design	23

7	Messungen	24
7.1	Low-Power Analyse und Optimierung	24
7.1.1	Reduktion Versorgungsspannung 3,3 V auf 3,0 V	24
7.1.2	Vergleich Energieverbrauch Aktiver Zyklus	24
7.1.3	Vergleich Energieverbrauch ESP32	26
7.1.4	Verluste Hardware	28
7.1.5	Vergleich Leistungsverbrauch Deep-Sleep	29
8	Fazit, Diskussion und Ausblick	31
8.1	Fazit	31
8.2	Kritische Beurteilung	31
8.3	Ausblick	33
A	PCB Schaltplan	34
B	Projektvereinbarung	37
C	Quellcode	51

1 Einleitung

Die zunehmende Vernetzung industrieller Prozesse stellt neue Anforderungen an eingebettete Systeme – insbesondere hinsichtlich Energieeffizienz, Messgenauigkeit und Kommunikationszuverlässigkeit. Diese Arbeit baut auf einem bestehenden Proof of Concept [4] auf und vertieft dessen technische Grundlagen mit Fokus auf die Weiterentwicklung der Smart-Waage selbst.

1.1 Ausgangslage

Ausgangslage für diese Arbeit bildet die Firma Skysec GmbH, ein Startup, das sich auf die Herstellung von Drohnen zum Schutz von Einrichtungen spezialisiert hat. Für den Herstellungsprozess, insbesondere die Inventarisierung von Kleinteilen und Schüttgut, sollen intelligente Waagen (Smart-Bins) eingesetzt werden. Bestehende kommerzielle Lösungen – etwa von Bossard – sind an proprietäre Cloud-Plattformen gebunden und erlauben keine eigenständige Datenhoheit, was für den Industriepartner aufgrund seiner sicherheitskritischen Tätigkeit nicht akzeptabel ist.

Im Rahmen der vorangegangenen Bachelorarbeit [4] (Berger & Müller, BA FS25) wurde bereits ein funktionsfähiger Prototyp entwickelt: ein ESP32-basiertes Wägesystem mit BLE-Kommunikation, einem Raspberry-Pi-Gateway und einem containerisierten ASP.NET-Core-Backend. Die vorliegende Arbeit knüpft an diesen PoC an und setzt den Schwerpunkt auf die Optimierung der Waage selbst – mit Fokus auf Messgenauigkeit und Energieverbrauch.

1.2 Zielsetzung

Die Zielsetzung gliedert sich in funktionale und organisatorische Ziele gemäss der Projektvereinbarung [8].

Tabelle 1: Funktionale Ziele gemäss Vereinbarung

Funktionales Ziel	Muss	Soll	Optional
Evaluation Hardware-Plattformen (ESP32, STM32, Arduino, Raspberry)	×		
Evaluation Kommunikationsprotokoll (BLE, ESP-NOW)	×		
Analyse Leistungsbedarf und Evaluation von Leistungssparmassnahmen	×		
Interface mit der Waage (Tara, Stückzahl, Gewicht, Einheitenumrechner)	×		
Power-Optimierung: Reduktion Aktiv- und Ruheströme	×		
Evaluation Energiequelle (Batterie vs. Akku)	×		
Temperaturkorrektur der Dehnmessstreifen		×	
Massenproduktionstaugliches PCB			×
Umporten auf STM32			×
Umbau auf Batteriebetrieb			×

Tabelle 2: Organisatorische Ziele gemäss Vereinbarung

Organisatorisches Ziel	Muss	Soll
Erstellung eines Projektplans	×	
Beschaffung der benötigten Hardware	×	
Einhaltung des Abgabetermins	×	
Einhaltung der Meilensteine		×

1.3 Systemabgrenzung und Randbedingungen

Folgende Bereiche sind explizit **nicht** Bestandteil dieser Arbeit:

- On-Premise-Lösung (keine Cloud-Anbindung)
- Dynamisches Softwareupdate (OTA)
- Einbindung in Firmennetzwerk
- Fertiges Produkt (Prototyp-Status)
- Verschlüsselte Übermittlung der Sensordaten an das Gateway
- Weiterentwicklung des bestehenden Backends

Randbedingungen:

- Arbeitsplätze im Raum TE514 / TE516 sowie Messgeräte, Lötstation und Industrieneetzgerät stehen bereit.
- Der Projektstand wird regelmässig mit dem Betreuer besprochen.
- Der Quellcode wird auf GitHub gehostet.

1.4 Meilensteine

1. **Hardware-Entwicklung:** Evaluation alternativer Plattformen; Redesign des Strompfads für Low-Power-Betrieb.
2. **Quality of Life:** Implementierung von Bedienelementen (Taster) sowie Software-Funktionen (Tara, Kalibrierung, Einheitenumrechner).
3. **Kommunikationsprotokolle:** Evaluation und Implementierung von ESP-NOW vs. BLE (NimbleBLE).

1.5 Aufbau der Arbeit

Die Arbeit ist in folgende Kapitel gegliedert: Kapitel 2 beschreibt die technische Ausgangslage und die getroffenen Designentscheidungen. In Kapitel 3 werden die Low-Power-Optimierungen erläutert – von der Hardwareanpassung bis zur Strommessung. Kapitel 4 behandelt die Optimierung der Messgenauigkeit, einschliesslich Sensorik, ADC-Ansteuerung und Kalibrierung. Eine Geräteübersicht beider Prototypen ist in Kapitel 5 dargestellt. Das PCB-Design ist in Kapitel 6 dokumentiert. Kapitel 7 fasst die Messungen zusammen, und Kapitel 8 schliesst mit Fazit und Ausblick.

2 Technische Ausgangslage

2.1 Design Space Exploration

2.1.1 MCU-Auswahl: SBC-NodeMCU-ESP32-C vs. ESP32-C3 vs. nRF52840

Für die Wahl des Mikrocontrollers wurden drei Kandidaten verglichen. Die wesentlichen Stromverbrauchswerte sind in Tabelle 3 gegenübergestellt.

MCU	Aktiv (mit RF)	Light Sleep	Deep Sleep
SBC-NodeMCU-ESP32-C (Board) [†]	160 mA – 240 mA	~5 mA	~4 mA
ESP32-C3 (SoC)	100 mA – 335 mA	130 μ A	5 μ A
nRF52840 (SoC)	~16 mA	1.5 μ A	0.4 μ A

Tabelle 3: Stromverbrauch der MCU-Kandidaten [6, 1, 5, 7]. [†] Board-Werte inkl. CP2102, LDO und Power-LED; ESP32-C3 und nRF52840 beziehen sich auf den SoC-Chip allein.

Das nRF52840 ist energetisch am effizientesten, erfordert jedoch ein vollständiges Porting der bestehenden Software sowie eine neue Peripherieschaltung. Der ESP32-C3 bietet auf SoC-Ebene einen deutlich besseren Sleep-Strom als das SBC-Entwicklungsboard und wäre mit moderatem Aufwand adaptierbar, benötigt aber ebenfalls eine eigene Peripheriebeschaltung. Da die Vorgängerarbeit [4] bereits auf dem SBC-NodeMCU-ESP32-C basiert, kann dessen bestehende Hardware- und Softwarebasis direkt übernommen werden, ohne eine Peripherieschaltung neu auslegen zu müssen. Angesichts des erheblichen Einsparpotenzials auf Seiten der DMS-Signalkette und der Software fiel die Entscheidung daher auf den SBC-NodeMCU-ESP32-C, um den Entwicklungsaufwand zu minimieren und den Fokus auf die relevanten Optimierungen zu legen. Der hohe Sleep-Strom des Boards – verursacht durch CP2102, Board-LDO und Power-LED – wurde dabei als bekannte Einschränkung in Kauf genommen.

3 Low-Power Optimierung

3.1 Motivation

Die Waage ist für den Einsatz in der Intralogistik konzipiert, wo sie autonom und ohne regelmässige Wartung den Füllstand von Lagerbehältern überwacht. In diesem Umfeld ist eine möglichst lange Batterielaufzeit entscheidend: Jeder Akkuwechsel verursacht Wartungsaufwand, der bei einer grossen Anzahl verteilter Geräte erhebliche Betriebskosten erzeugen kann.

Da die Vorgängerarbeit ihren Schwerpunkt auf die funktionale Korrektheit des Systems legte, wurde die Optimierung des Stromverbrauchs bisher nicht adressiert. Das System bietet daher noch erhebliches Einsparpotenzial, das in dieser Arbeit systematisch erschlossen wird.

3.2 Strompfad Schema

Die folgenden Diagramme zeigen den Strompfad der alten und neuen Waage.

3.2.1 Ausgangslage

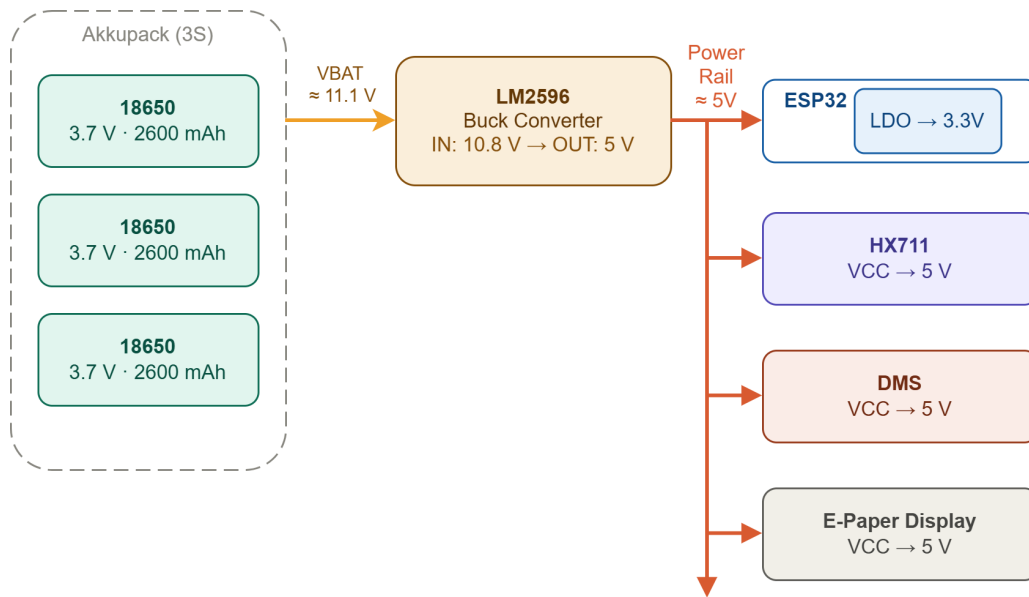


Abbildung 1: Schema Strompfad – Ausgangslage

Die Spannungsversorgung der Ausgangslage besteht aus drei in Reihe geschalteten 18650-Lithium-Ionen-Zellen mit einer Nennspannung von 11.1 V. Ein **LM2596**-Schaltregler wandelt diese Spannung auf ein stabiles 5 V-Rail herunter, von dem alle Verbraucher gespeist werden: der ESP32 (über den internen LDO des Entwicklerboards), der HX711-ADC, die Dehnmessstreifen sowie das E-Paper-Display. Der LDO des Entwicklerboards wandelt die 5 V auf die vom Mikrocontroller benötigten 3.3 V – eine zusätzliche Spannungsconversion, die unnötige Verlustleistung erzeugt und im Hardware Redesign beseitigt wird.

3.2.2 Hardware Redesign

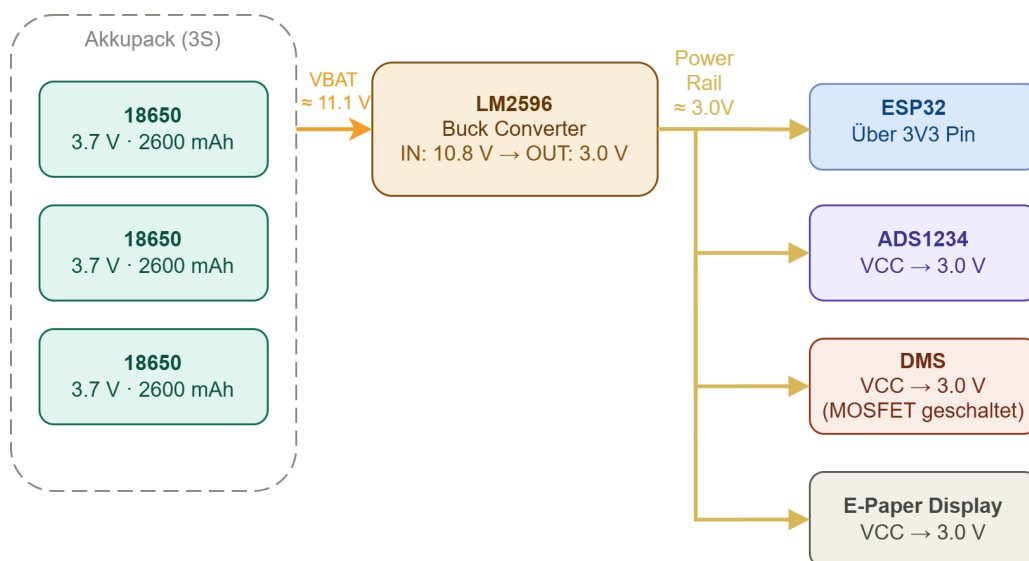


Abbildung 2: Schema Strompfad – Hardware Redesign

Das Hardware Redesign optimiert den Strompfad in zwei wesentlichen Punkten:

- **Dehnmessstreifen über MOSFET geschaltet:** In der Ausgangslage waren die Wägezellen dauerhaft mit Spannung versorgt – auch im Deep Sleep. Durch einen MOSFET wird die Versorgung der Dehnmessstreifen vor dem Eintreten in den Deep Sleep unterbrochen, was den Ruhestrom merklich senkt.
- **ESP32 direkt über den 3V3-Pin versorgt:** Das Entwicklerboard enthielt einen internen LDO, der 5 V auf 3.3 V regelte. Im neuen Design wird der ESP32 direkt über den 3V3-Pin eingespeist, womit diese Verlustleistung entfällt. Ausserdem wurde die Ausgangsspannung des Buck-Converters auf 3.0 V reduziert, was gegenüber dem Betrieb mit 3.3 V eine weitere Einsparung von 73.4 mJ pro Zyklus (−17,6 %) erzielt (siehe Abschnitt 7.1.1).

3.3 Strommessungen und Low-Power-Optimierung

Ein zentrales Ziel dieser Arbeit ist die Minimierung des Stromverbrauchs, um eine möglichst lange Batterielaufzeit zu erreichen. Die folgenden Messungen dokumentieren den Energieverbrauch in den einzelnen Betriebsphasen und quantifizieren den Effekt der implementierten Optimierungen. Tabellen 6 und 4 fassen die verbrauchte Energie pro Phase zusammen – jeweils im direkten Vergleich zwischen Ausgangslage und optimiertem System.

3.4 Übersicht Zyklus

Für die Strommessungen kamen zwei Messgeräte zum Einsatz, die je nach Messaufgabe in den Versorgungspfad eingeschleift wurden.

Das **Nordic Semiconductor Power Profiler Kit II** (PPK2) wurde für das qualitative Stromverlauf-Profiling eingesetzt und liefert eine zeitlich aufgelöste Visualisierung der einzelnen Betriebsphasen (Boot, ADC-Messung, BLE, Deep Sleep) in der nRF Connect Power Profiler App.

Die quantitativen Energie- und Leistungsmessungen wurden mit dem **JouleScope** durchgeführt. Das JouleScope misst gleichzeitig Strom und Spannung und berechnet daraus direkt die verbrauchte Energie in Millijoule – unter Berücksichtigung des tatsächlichen Spannungsniveaus am jeweiligen Messpunkt. Je nach Messung wurde das Gerät an unterschiedlichen Stellen in den Strompfad eingeschleift: vor dem Buck-Converter (Batterieseite), nach dem Buck-Converter (Power-Rail), direkt vor einzelnen Verbrauchern (ESP32, ADC, Dehnmessstreifen) oder am LDO-Ein- bzw. Ausgang.

3.5 Gesamtsystem Stromverbrauch

Tabelle 6 vergleicht die Software-Ausgangslage mit dem optimierten System auf Basis der JouleScope-Messungen am 3V3-Pin des ESP32. Tabelle 4 stellt den Gesamtenergieverbrauch beider Systeme gegenüber; die Werte beziehen sich auf JouleScope-Messungen vor dem Buck-Converter (Batterieseite) und umfassen damit sämtliche Wandlungsverluste.

3.5.1 Vergleich Energieverbrauch Aktiver Zyklus

Tabelle 4: Gesamtsystem Energiebedarf Zusammensetzung – Vergleich Aktiver Zyklus

Verbraucher	Alt [mJ]	Neu [mJ]	Ersparnis
<i>Verbrauch</i>			
ESP32 über 3V3	2220	223.8	−89.9 %
Dehnmessstreifen	781.9	89.1	−88.6 %
ADC	106.2	13.42	−87.4 %
<i>Verluste Umwandlung</i>			
Verlust LDO	1210	0	Entfällt
Verlust Buck	2955	346.1	−88.3 %
Summiert	7273.1	672.4	−90.7 %
Diskrepanz	−770.1	17.1	
Gesamt Gemessen	6503	689.5	−89.4 %

3.5.2 Vergleich Leistungsverbrauch Deep-Sleep

Tabelle 5 vergleicht den Leistungsverbrauch im Deep Sleep. Einzelmessungen der Verbraucher: Abb. 36 (DMS), 37/38 (ADC). Gesamtmessungen vor dem Buck-Converter: Abb. 39/40. Trotz der Komponenteneinsparungen (−87,7 % nach Buck) begrenzt der Leerlaufverlust des LM2596S von 69.77 mW (Abb. 41) die tatsächliche Ersparnis auf Batterieebene erheblich: Er übersteigt allein bereits den Gesamtverbrauch aller Komponenten des neuen Systems.

Tabelle 5: Vergleich Leistungsverbrauch Deep-Sleep – Ausgangslage vs. Optimiertes System

Verbraucher	Alt [mW]	Neu [mW]	Ersparnis
<i>Nach Buck-Converter (Power Rail)</i>			
Dehnmessstreifen	78,19	0	−100 % (MOSFET)
ADC	9,762	4,042	−58,6 %
ESP32	27,53	10,15	−63,1 %
Summiert (nach Buck)	115,4	14,19	−87,7 %
<i>Vor Buck-Converter (Batterieseite)</i>			
Gesamt gemessen	268,2	90,96	−66,1 %

3.6 Vergleich Energieverbrauch ESP32

3.6.1 Übersicht Stromverlauf ESP32

Abbildung 3 zeigt den Stromverlauf eines vollständigen Messzyklus in der Ausgangslage, wie er direkt aus der Vorgängerarbeit übernommen wurde. In dieser Vorarbeit stand die funktionale Korrektheit im Vordergrund – eine gezielte Optimierung des Stromverbrauchs hatte keine Priorität.

Im Profil sind drei charakteristische Abschnitte erkennbar: ein kurzer initialer Peak während Boot und Initialisierung, ein ruhigeres Plateau während der ADC-Messung sowie ein erhöhter und zeitlich ausgedehnter Abschnitt während der BLE-Kommunikation. Die gesamte Aktivphase erstreckt sich über ca. **10 s**, bevor das System in den Deep Sleep übergeht.

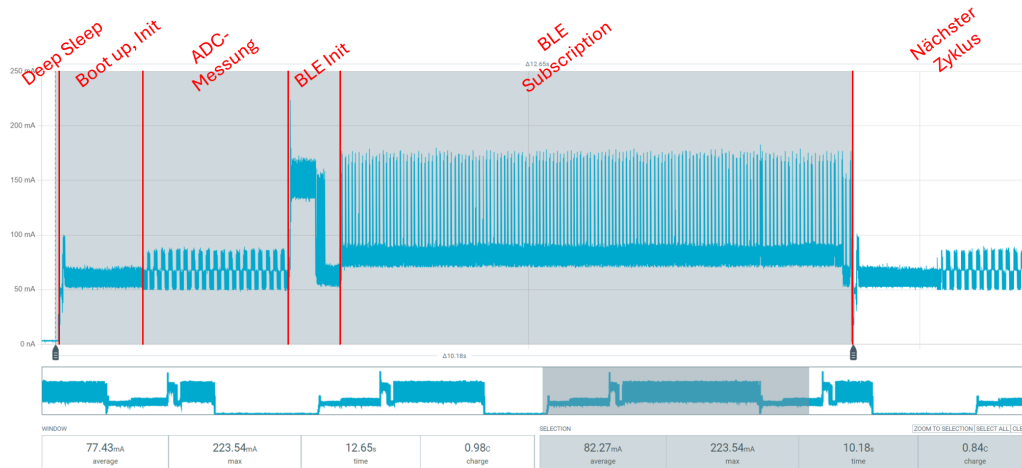


Abbildung 3: Stromverbrauch – Ausgangslage, ein vollständiger Messzyklus

1. **Deep Sleep:** Zwischen den Messzyklen befindet sich das System im Deep Sleep. Die meisten Funktionen sind abgeschaltet; lediglich der RTC-Timer läuft weiter und weckt den ESP32 nach Ablauf des konfigurierten Intervalls. Der Stromverbrauch ist in dieser Phase minimal.
2. **Boot und Initialisierung:** Der ESP32 startet nach dem Deep Sleep neu. Der nichtflüchtige Speicher (NVS) wird ausgelesen, die State Machine initialisiert und der HX711-ADC eingerichtet.
3. **ADC-Messung:** Der HX711 erfasst 20 Samples auf einem einzigen Kanal, an dem alle vier Wägezellen als gemeinsame Brücke angeschlossen sind. Zwischen jedem Sample wartet der ESP32 in einem blockierenden `delay(50ms)`, was den Kern während der gesamten Messphase vollständig aktiv hält.
4. **BLE-Initialisierung:** Der BLE-Stack wird gestartet und das Advertisement für die Gateway-Verbindung gesendet.
5. **Verbindung und Datenübertragung:** Das System wartet auf den Verbindungsaufbau des Gateways (typisch ca. 6 s), überträgt den Messwert per GATT-Indication und trennt die Verbindung.
6. **Nächster Zyklus:** Der Messzyklus ist abgeschlossen; das System kehrt in den Deep Sleep zurück. Im gezeigten Stromprofil folgt der nächste Zyklus sofort, weshalb kein ausgeprägtes Tiefstromplateau sichtbar ist.

3.6.2 Vergleich ESP32 Energieverbrauch

Tabelle 6: Software-Vergleich Aktiver Zyklus am ESP32-3V3-Pin: Ausgangslage vs. Optimiertes System. Beide ESP32 wurden mit 3.0V gespeist. DMS und LDO sind aus dem Messpfad ausgeschlossen. Messprofile und Berechnungen vgl. Abschnitt 7.1.3.

Betriebsphase	Alt [mJ]	Neu bereinigt* [mJ]	Einsparung
Boot und Ramp-up	360.3	57.66	−84.0 %
ADC-Messung	807.8	37.41	−95.4 %
BLE	1293	88.18	−93.2 %
ADS1234 Chip (Abzug*)	–	13.42	–
Gesamt Zyklus	1858	210.4	−88.7 %

* Der ADS1234 ist am ESP32-3V3-Rail angeschlossen und in den Phasenmessungen des neuen Systems enthalten. Da der HX711 des alten Systems ausserhalb des Messpfades lag, wird der ADS1234-Anteil (13.42 mJ) von allen Neu-Werten abgezogen. Rohmessungen und Rechnungen: Abschnitt 7.1.3.

Die nachfolgenden Abschnitte beschreiben jede Betriebsphase im Detail und erläutern, welche Massnahmen zur jeweiligen Einsparung geführt haben:

3.6.3 Optimierung Energieverbrauch Software

1. Ramp Up – Boot, Initialisierung und ADC:

Diese Phase umfasst den ESP32-Bootvorgang, die State-Machine-Initialisierung sowie die ADS1234-Messung. In der Vorgängerarbeit wurden Kalibrierparameter bei jedem Aufwachen aus dem Deep Sleep neu aus dem nichtflüchtigen Speicher (NVS) ausgelesen. Im optimierten System werden diese Werte einmalig beim ersten Boot geladen und anschliessend im RTC-Speicher gehalten, der den Deep Sleep überlebt – das NVS-Auslesen entfällt damit bei jedem Folge-Zyklus und verkürzt die Boot-Phase entsprechend.

2. ADC:

In der Ausgangslage verwendete die Vorgängerarbeit den **HX711** – einen einkanali- gen 24-Bit-ADC, an den alle vier Wägezellen als gemeinsame Brücke angeschlossen waren. Die Messung erfasste 20 Samples auf diesem einzigen Kanal; zwischen jedem Sample wurde ein blockierendes `delay(50ms)` ausgeführt, was den ESP32-Kern für die gesamte Wartezeit vollaktiv liess.

Im optimierten Design werden die vier Kanäle des ADS1234 sequenziell ausgelesen; pro Kanal werden **5 Samples** erfasst. Nach jedem Sample kehrt der ESP32 sofort in den Light Sleep zurück und wird durch einen GPIO-Interrupt auf dem `DRDY`-Pin geweckt. Zwischen den vier Kanalgruppen liegt eine Pause von ca. **400 ms**: die physikalisch bedingte Einschwingzeit des internen sinc^3 -Filters nach einem Kanalwechsel. Auch diese Wartezeit verbringt der ESP32 im Light Sleep.

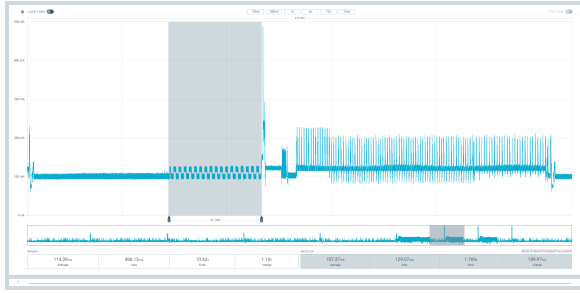


Abbildung 4: ADC-Stromverlauf – Ausgangslage (HX711)

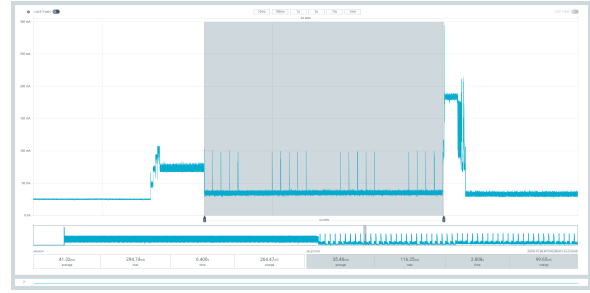


Abbildung 5: ADC-Stromverlauf – Optimiertes System (ADS1234)

3. BLE-Initialisierung und Advertisement:

In der Ausgangslage verwendete die Firmware die Arduino-BLE-Bibliothek (Bluedroid) und baute für jeden Messzyklus eine vollständige GATT-Verbindung auf. Die Dauer dieser Phase variiert stark in Abhängigkeit vom Scan-Intervall des Gateways – im gezeigten Profil **5.6 s**, in anderen Messungen bis zu **7 s**.

Der fundamentale Optimierungsansatz bestand darin, den Verbindungsaufbau vollständig zu eliminieren: Der Messwert wird direkt in die Manufacturer-Specific-Data des Advertisement-Pakets kodiert – das Gateway empfängt die Daten durch passives Scannen, ohne eine Verbindung aufzubauen. Im zweiten Schritt wurde Bluedroid durch den schlanken **NimBLE**-Stack ersetzt, der einen deutlich geringeren Initialisierungs-overhead aufweist.

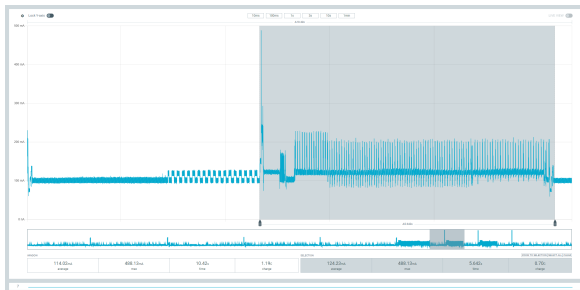


Abbildung 6: BLE – Ausgangslage (Bluedroid, GATT-Verbindung)

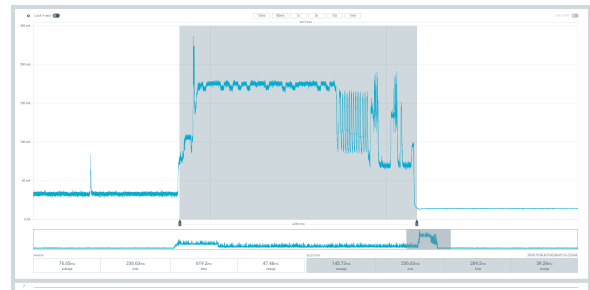


Abbildung 7: BLE – Optimiertes System (NimBLE, Advertisement)

Als Zwischenschritt wurde der Advertisement-Ansatz zunächst mit der bestehenden Arduino-BLE-Bibliothek validiert (Abbildung 8), bevor der Wechsel zu NimBLE erfolgte.

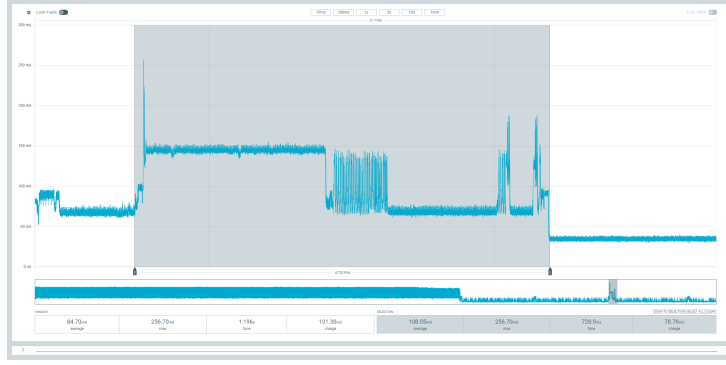


Abbildung 8: BLE – Zwischenschritt (Arduino BLE-Library, Advertisement)

Die Wahl von **zwei Advertisements** pro Zyklus ist ein bewusster Kompromiss. In einem Zuverlässigkeitstest über **100 aufeinanderfolgende Messzyklen** wurden **91 von 100** Advertisements erfolgreich vom Gateway empfangen, was für ein periodisches Überwachungssystem ausreichend ist.

```

info: EdgeDevice.BLE.GWBackgroundService[0]
      Advertisement #324 received. Window: 91/100 received (9 missed). Total: 265 received, 27 missed.
info: EdgeDevice.BLE.GWBackgroundService[0]
      Advertisement weight received: -114976.74 g from 14:2B:2F:EC:1A:EA
  
```

Abbildung 9: Zuverlässigkeitstest – 100 BLE Advertisements (NimBLE)

3.7 Verluste Hardware

Dieses Unterkapitel quantifiziert die Wandlungsverluste des Buck-Converters und des LDO sowie den Energiebedarf des ADC.

3.7.1 Messmethode

Für Buck-Converter und LDO wird die Energie jeweils *vor* und *nach* dem Bauteil über einen vollständigen Zyklus erfasst; die Differenz entspricht dem jeweiligen Wandlungsverlust. Der Energiebedarf des ADC wird direkt am Versorgungspin des Chips gemessen.

3.7.2 Buck converter

Zur Bestimmung des Verlusts am LM2596S-Schaltregler wird die Energie *vor* dem Konverter (Eingang, Batterieseite) mit der Energie *nach* dem Konverter (Ausgang, Lastseite) verglichen; die Differenz entspricht dem Schaltregler-Board zuzuschreibenden Verlust. Die Messwerte entstammen den JouleScope-Aufzeichnungen (Abbildungen 30, 31, 32 und 33) und umfassen jeweils einen vollständigen Messzyklus. Beim alten System wurde sichergestellt, dass BLE-Initialisierung und Subscription zusammen ca. 5.4 s dauerten, um vergleichbare und reproduzierbare Messungen zu gewährleisten.

Altes System

$$\Delta E_{\text{Buck, alt}} = 6503 \text{ mJ} - 3548 \text{ mJ} = 2955 \text{ mJ} \quad (1)$$

$$\eta_{\text{Buck, alt}} = \frac{3548 \text{ mJ}}{6503 \text{ mJ}} \approx 54.6 \% \quad (2)$$

Neues System

$$\Delta E_{\text{Buck, neu}} = 689.5 \text{ mJ} - 343.4 \text{ mJ} = 346.1 \text{ mJ} \quad (3)$$

$$\eta_{\text{Buck, neu}} = \frac{343.4 \text{ mJ}}{689.5 \text{ mJ}} \approx 49.8 \% \quad (4)$$

Vergleich Beide Systeme weisen einen ähnlichen Wirkungsgrad von ca. 50 % auf. Der entscheidende Unterschied liegt im absoluten Verlust: Das alte System verliert 2955 mJ pro Zyklus am Schaltregler, das neue System nur 346.1 mJ – eine Reduktion um den Faktor **8,5**. Der geringere absolute Verlust ist direkt auf den deutlich niedrigeren Gesamtenergieverbrauch des optimierten Systems zurückzuführen.

Plausibilitätsprüfung anhand des Datenblatts Das Datenblatt des LM2596S [10] gibt für eine Konfiguration von 12 V Eingang auf 3.3 V Ausgang bei einem Ausgangsstrom von 3 A einen typischen Wirkungsgrad von 79 % an. Die gemessenen Werte von ca. 50 % liegen erwartungsgemäss darunter – zum einen weil beide Systeme mit mittleren Lastströmen weit unter 3 A betrieben werden, zum anderen weil es sich nicht um den nackten LM2596S-Chip handelt, sondern um ein fertiges Schaltregler-Board mit zusätzlicher Peripherie (Freilaufdiode, Spule, Filterkapazitäten, Schutzschaltungen), die den Gesamtwirkungsgrad weiter senken.

Bei geringer Nutzlast dominiert zudem der Quiescent-Strom des LM2596S von ca. 5.5 mA den Eingangsstrom überproportional – bei 3 A Nennlast wäre er vernachlässigbar. Da der LM2596S keinen Burst- oder PFM-Modus für Leichtlast unterstützt, ist dieser Verlust strukturell bedingt.

3.7.3 LDO

Der interne LDO des ESP32-Entwicklerboards wandelt die vom LM2596S gelieferte Spannung auf die vom Mikrocontroller benötigten 3.3 V. Diese zusätzliche Konverterstufe erzeugt eine nicht vernachlässigbare Verlustleistung.

Ausgangslage Die Energie vor dem LDO (am VIN-Pin, Eingang) beträgt 3430 mJ; nach dem LDO (am 3V3-Pin, Ausgang) verbleiben 2220 mJ:

$$\Delta E_{\text{LDO, alt}} = 3430 \text{ mJ} - 2220 \text{ mJ} = 1210 \text{ mJ} \quad (5)$$

$$\eta_{\text{LDO, alt}} = \frac{2220 \text{ mJ}}{3430 \text{ mJ}} \approx 64.7 \% \quad (6)$$

Die Messprofile sind in Abbildungen 34 und 35 dargestellt.

Optimiertes System – LDO entfällt Im optimierten System wird der ESP32 direkt über den 3V3-Pin eingespeist, anstatt über den VIN-Pin. Damit wird der interne LDO vollständig umgangen, und der Verlust von 1210 mJ pro Zyklus entfällt ersatzlos (vgl. Tabelle 4, Zeile *Verlust LDO*).

3.7.4 ADC

Der ADS1234 wird nach der Messung über den !PDWN-Pin in den Standby-Modus versetzt und vor dem nächsten Zyklus wieder aktiviert. Da der ESP32 im Anschluss in den Deep Sleep übergeht und die GPIO-Pins floatend werden, bleibt der ADC im Tiefschlaf teilweise aktiv; ein Pull-down-Widerstand am !PDWN-Pin hätte eine vollständige Abschaltung ermöglicht. Im alten System war der HX711 hardwareseitig nicht im Messpfad eingebunden und wird daher separat betrachtet. Die Zahlenwerte sind Tabelle 4 zu entnehmen; die Messprofile sind in Abbildungen 10 und 11 dargestellt.

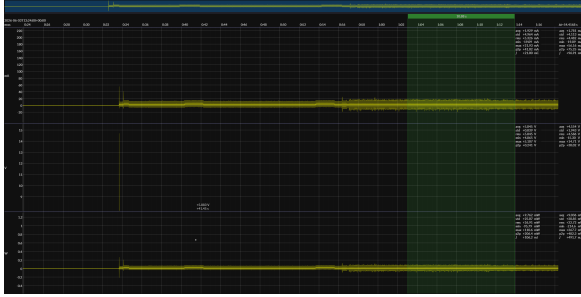


Abbildung 10: ADC-Verlust – Ausgangslage (HX711); 9.765 mW



Abbildung 11: ADC-Verlust – Optimiertes System (ADS1234); 3.738 mW

3.8 Gesamtverbrauch und Batterielebensdauer

Die Batterielaufzeit ergibt sich aus der Energiebilanz pro Messzyklus; Grundlage sind die direkt am JouleScope gemessenen Werte vor dem Buck-Converter (Batterieseite):

- **Aktivzyklus-Energie:** 689.5 mJ (neu) vs. 6503 mJ (alt), siehe Tabelle 4
- **Deep-Sleep-Leistung:** 90.96 mW (neu) vs. 268.2 mW (alt), siehe Tabelle 5

Als Batteriekapazität wird 3×2600 mA h bei 11.1 V angenommen:

$$E_{\text{Bat}} = 103.9 \text{ kJ}$$

Die Energie pro Messzyklus ergibt sich zu (da $t_{\text{aktiv}} \ll T$):

$$E_{\text{Zyklus}}(T) \approx E_{\text{aktiv}} + P_{\text{sleep}} \cdot T$$

Tabelle 7: Berechnete Batterielaufzeit – Vergleich Alt vs. Neu (3×2600 mA h, 11.1 V)

Intervall T	$E_{\text{Zyklus, neu}}$	Laufzeit neu	$E_{\text{Zyklus, alt}}$	Laufzeit alt
1 min (60 s)	6.15 J	$\approx$ 12 Tage	22.6 J	$\approx$ 3 Tage
10 min (600 s)	55.3 J	$\approx$ 13 Tage	167 J	$\approx$ 4 Tage
30 min (1800 s)	164 J	$\approx$ 13 Tage	489 J	$\approx$ 4 Tage

Die Laufzeit ist nahezu intervallunabhängig, da der Deep-Sleep-Ruhestrom die Energiebilanz dominiert. Das grösste verbleibende Einsparpotenzial liegt in der Reduktion des Tiefschlaf-Ruhestroms – primär durch Ersetzen des SBC-NodeMCU-ESP32-C-Boards und des LM2596S.

4 Optimierung Messgenauigkeit

4.1 Sensorik: Wägezellen und ADC

4.1.1 Wägezellen (YZC-133)

Als Kraftsensoren kommen vier Wägezellen des Typs YZC-133 zum Einsatz [2]. Jede Zelle ist für eine Nennlast von 1 kg ausgelegt und enthält intern eine vollständige Wheatstone-Brücke aus vier Dehnungsmessstreifen (DMS). Die relevanten elektrischen Kenngrößen sind in Tabelle 8 zusammengefasst.

Tabelle 8: Elektrische Spezifikation YZC-133

Parameter	Wert
Nennlast	1 kg
Nennkennwert (Empfindlichkeit)	$1,0 \pm 0,15 \text{ mV/V}$
Nullpunktsignal	$\pm 0,1 \text{ mV/V}$
Kriechfehler	$0,03 \% \text{ F.S./30 min}$
Eingangsimpedanz	$1066 \pm 20 \Omega$
Ausgangsimpedanz	$1000 \pm 5 \Omega$

Bei 3.0 V Erregerspannung liegen beide Ausgangspins der Brücke auf einem Gleichtakt-potenzial von ca. 1.5 V ($V_{\text{ex}}/2$). Das Nutzsignal ist die *Differenz* zwischen diesen beiden Pins: bei Vollast nominell $1,0 \text{ mV/V} \times 3,0 \text{ V} = 3,0 \text{ mV}$. Der differentielle Eingang des ADS1234 erfasst genau diese Differenz und unterdrückt das Gleichtaktsignal durch seine hohe Gleichtaktunterdrückung (CMRR). Die vier Zellen sind in X-Formation an den Ecken der Waagplattform montiert.

4.1.2 ADS1234 – 4-Kanal-24-Bit-ADC

Für die Signalerfassung wird der ADS1234 von Texas Instruments eingesetzt [9]. Der Chip ist ein 24-Bit-Sigma-Delta-ADC mit vier differentiellen Eingangskanälen und einem integrierten programmierbaren Verstärker (PGA) mit den Verstärkungsfaktoren 1, 2, 64 und 128. Mit PGA-Gain 128 ergibt sich für das 3.0 mV-Signal der YZC-133 ein verstärktes Differenzsignal von ca. 384 mV, das den Eingangsbereich des ADC gut ausnutzt. Die wählbare Ausgabedatenrate beträgt 10 SPS oder 80 SPS; im vorliegenden System wird 10 SPS verwendet, da dies das Eingangsrauschen durch die längere Integrationszeit maximal reduziert.

4.1.3 Von einem auf vier ADC-Kanäle

In der Vorarbeit zu dieser Bachelorarbeit waren alle vier Wägezellen in einer gemeinsamen Wheatstone-Brücke zusammengeschaltet und lieferten ihr summiertes Signal an einen einzigen ADC-Kanal. In dieser Arbeit wird jede Wägezelle einzeln an einen eigenen Eingangskanal des ADS1234 geführt. Dieser Wechsel bringt zwei wesentliche Vorteile:

Ecklastkompensation Mit einem einzigen Summensignal ist keine positionsabhängige Korrektur möglich: Das System kann nicht unterscheiden, ob eine Abweichung durch

einen exzentrischen Lastangriffspunkt oder durch eine tatsächliche Massenänderung entsteht. Durch die individuellen Kanäle lässt sich jede Wägezelle separat kalibrieren und der dominante Sensor pro Lastposition gezielt korrigieren (siehe Abschnitt 4.2).

Rauschverhalten und individuelle Kalibrierung Die Empfindlichkeit der YZC-133 streut mit $\pm 15\%$ zwischen den Exemplaren. In der Einzelkanal-Lösung mittelte sich diese Streuung über die parallele Zusammenschaltung heraus, konnte aber nicht pro Sensor korrigiert werden. Mit vier separaten Kanälen wird die Empfindlichkeitsdifferenz jeder Wägezelle durch den individuell bestimmten Skalierungsfaktor (`spanFactor`) in der Software exakt kompensiert.

4.2 ADS1234 – Ansteuerung und Kalibrierung

4.2.1 Überblick

Der ADS1234 ist ein 24-Bit-Sigma-Delta-ADC von Texas Instruments, der speziell für den Einsatz mit Brückensensoren wie Dehnungsmessstreifen (DMS) ausgelegt ist [9]. Er stellt vier unabhängige Eingangskanäle bereit, die über zwei digitale Adressleitungen (A0, A1) ausgewählt werden. Die Kommunikation mit dem Mikrocontroller erfolgt über ein synchrones serielles Protokoll mit den Leitungen `DRDY/DOUT` (kombiniertes Datenbereit- und Datensignal), `SCLK` (Takt) sowie `!PDWN` (Power-Down/Reset).

Tabelle 9: Pin-Belegung ADS1234 – ESP32

Signal	GPIO	Funktion
<code>DRDY/DOUT</code>	2	Datenbereit-Anzeige und serieller Datenausgang
<code>SCLK</code>	15	Serieller Takt (Master-Ausgang)
<code>!PDWN</code>	26	Power-Down / Reset
<code>DMS_PWR</code>	17	Versorgung der Wägezellen (schaltbar)
<code>A0</code>	32	Kanal-Adresse Bit 0
<code>A1</code>	4	Kanal-Adresse Bit 1

4.2.2 Serielles Leseprotokoll

Der ADS1234 signalisiert eine abgeschlossene Wandlung, indem er die `DRDY`-Leitung auf `LOW` zieht. Die Software wartet aktiv auf diesen Zustand und liest anschliessend 24 Bit MSB-first aus, indem für jedes Bit ein `SCLK`-Impuls erzeugt und der Pegel von `DOUT` abgetastet wird. Nach dem 24. Bit wird gemäss Datenblatt (Figure 7-10) ein 25. Taktimpuls gesendet, der `DRDY` wieder auf `HIGH` zwingt und damit den ADC für die nächste Wandlung freigibt. Da der ADS1234 ein Zweierkomplement-Format mit 24 Bit Breite verwendet, wird der Rohwert abschliessend vorzeichenrichtig auf 32 Bit erweitert:

```
if (raw & 0x800000) raw |= 0xFF000000;
return (int32_t)raw;
```

4.2.3 Kanalwechsel CH4 → CH1: !PDWN-Workaround

Der ADS1234 kodiert den aktiven Kanal über die beiden Adressleitungen als 2-Bit-Binärzahl: Kanal 1 entspricht A0=0, A1=0, Kanal 4 entspricht A0=1, A1=1. Der Chip erkennt einen Kanalwechsel intern anhand steigender Flanken auf A0 oder A1. Beim Wechsel von Kanal 4 nach Kanal 1 fallen jedoch *beide* Adressleitungen von HIGH auf LOW – es entsteht keine einzige steigende Flanke. Der ADS1234 erkennt diesen Wechsel daher nicht als Kanalschaltung und setzt DRDY nicht zurück. Die Leitung bleibt LOW, was der Software fälschlicherweise signalisiert, dass bereits ein gültiger Messwert des neuen Kanals vorliegt, obwohl noch der alte Kanal aktiv ist.

Als Lösung wird nach dem Setzen der Adressleitungen auf Kanal 1 geprüft, ob DRDY bereits LOW ist. Trifft dies zu, wird ein !PDWN-Puls von 500 µs ausgelöst, der den ADC in den Power-Down-Zustand versetzt und anschliessend neu startet. Dieser Reset zwingt den Chip, den ausgewählten Kanal neu zu initialisieren und eine frische Wandlung durchzuführen, bevor DRDY das nächste Mal LOW wird.

```
int32_t readChannel(int ch) {
    setChannel(ch);
    if (ch == 1 && digitalRead(PIN_DRDY_DOUT) == LOW) {
        digitalWrite(PIN_PDWN, LOW);
        delayMicroseconds(500);
        digitalWrite(PIN_PDWN, HIGH);
    }
    return readADS1234();
}
```

4.2.4 Kalibrierung

Die Kalibrierung läuft in drei aufeinanderfolgenden Schritten ab und wird einmalig beim Start der Firmware über die serielle Schnittstelle ausgelöst. Alle Messwerte sind Mittelwerte aus `CAL_SAMPLES = 5` Wandlungen pro Kanal, was bei 10 SPS einer Messzeit von rund 0,4 Sekunden pro Kanal entspricht.

Schritt 1 – Nullpunkt (Zero Offset) Die Waage wird vollständig entlastet und für jeden der vier Kanäle ein gemittelter Rohwert erfasst, der als Nullpunkt `zeroOffset[ch]` gespeichert wird. Dieser Wert kompensiert thermische und mechanische Vorlast der Wägezellen sowie Offset-Fehler des ADC.

Schritt 2 – Empfindlichkeit (Span) Ein bekanntes Kalibriergewicht von 3000 g wird mittig auf die Waagplattform gelegt. Für jeden Kanal wird die Abweichung vom Nullpunkt berechnet:

$$\Delta_i = \bar{x}_i - \text{zeroOffset}_i \quad (7)$$

Die vier Wägezellen sind in X-Formation an den Ecken der Plattform angebracht. Bei dieser Anordnung werden die zwei diagonal gegenüberliegenden Zellen beim Belasten auf Zug, die anderen zwei auf Druck beansprucht, was zu einem mechanisch bedingten Vorzeichenwechsel im ADC-Signal führt. Das Vorzeichen von Δ_i wird daher pro Kanal automatisch

erfasst und als `channelSign` gespeichert. Der initiale Skalierungsfaktor ergibt sich aus der Gesamtauslenkung über alle vier Kanäle:

$$s_{\text{init}} = \frac{m_{\text{cal}}}{\sum_{i=1}^4 |\Delta_i|} \quad \Rightarrow \quad \text{spanFactor}_i = \text{channelSign}_i \cdot s_{\text{init}} \quad (8)$$

Damit trägt jeder Kanal anteilig zur Gesamtmasse bei, wobei die Polarität korrekt berücksichtigt wird.

Schritt 3 – Ecklastkompensation (Corner Trim) Bei einer Plattformwaage mit vier Wägezellen in den Ecken führen fertigungsbedingte Toleranzen der Sensoren dazu, dass ein exzentrisch aufgelegtes Gewicht – je nach Position – unterschiedliche Gesamtmassen anzeigt. Um dies zu kompensieren, wird das Kalibriergewicht nacheinander über jede der vier Ecken gestellt.

Bei jeder Eckenbelastung ermittelt die Software denjenigen Kanal mit der grössten absoluten Auslenkung (*dominanter Kanal*), da dieser am stärksten von der Lastposition beeinflusst wird. Die aktuelle Gesamtanzeige m_{meas} ergibt sich aus den Beiträgen aller vier Kanäle:

$$m_{\text{meas}} = \sum_{i=1}^4 \text{spanFactor}_i \cdot \Delta_i \quad (9)$$

Der Skalierungsfaktor des dominanten Kanals d wird anschliessend so angepasst, dass die Anzeige exakt dem Kalibriergewicht entspricht, während die Faktoren der übrigen Kanäle unverändert bleiben:

$$\text{spanFactor}_d = \frac{m_{\text{cal}} - \sum_{i \neq d} \text{spanFactor}_i \cdot \Delta_i}{\Delta_d} \quad (10)$$

Ein einziger Durchlauf über alle vier Ecken ist für typische DMS-Toleranzen von unter 5 % ausreichend.

Gewichtsberechnung im Betrieb Nach abgeschlossener Kalibrierung berechnet sich die angezeigte Masse in jedem Messtakt als:

$$m = \sum_{i=1}^4 \text{spanFactor}_i \cdot (\text{raw}_i - \text{zeroOffset}_i) \quad (11)$$

4.3 Messreihe Messgenauigkeit

Zur Beurteilung der Messgenauigkeit wurden an sechs Messpositionen (vier Ecken sowie die Mitte der Wägefläche) jeweils 5 aufeinanderfolgende Messwerte aufgezeichnet. Die Ergebnisse sind in Tabelle 10 zusammengefasst.

Präzision vs. Genauigkeit. Die Wiederholbarkeit der Messung ist durchgehend hoch: Die Streuung innerhalb jeder Messreihe beträgt lediglich ca. 0,1 g bis 0,2 g, was auf eine stabile und rauscharme Signalverarbeitung hinweist. Die absolute Genauigkeit hingegen weist erhebliche Abweichungen auf. Besonders auffällig ist die Mittenbelastung mit 3 kg, bei der ein Messwert von rund 3206 g statt 3000 g ausgegeben wird – eine Abweichung von über 6 %.

Position	Referenz	Messwerte (g)	Mittelwert (g)	Abweichung (g)
Ecke 1	1000,0 g	1020,5 – 1020,7	1020,6	+20,6
Ecke 2	1000,0 g	975,4 – 975,7	975,5	–24,5
Ecke 3	1000,0 g	996,6 – 996,7	996,6	–3,4
Ecke 4	1000,0 g	999,9 – 1000,2	1000,0	$\pm 0,0$
Mitte	1000,0 g	1033,2 – 1035,6	1035,1	+35,1
Mitte	3000,0 g	3205,6 – 3205,8	3205,6	+205,6

Tabelle 10: Messreihe optimiertes Design: Messwerte an verschiedenen Positionen

Mechanische Einflüsse. Neben der Kalibrierung tragen auch mechanische Unzulänglichkeiten des Prototyps zum Messfehler bei. Die vier Wägezellen sind nicht exakt symmetrisch verbaut, da die Waage mit Heisskleber fixiert wurde und dadurch geringfügig in der Horizontalen geneigt ist. Zusätzlich streift die Wägezelle an Ecke 4 leicht den Rand des 3D-gedruckten Gehäuses, was deren Messwert mechanisch beeinflusst. Diese Effekte erklären, weshalb die positionsabhängigen Abweichungen stark variieren und keine konsistente Offset-Korrektur durch einfache Nachkalibrierung möglich ist.

Fehlende Transientunterdrückung. Ein weiteres Problem tritt beim Aufbringen oder Abnehmen eines Gewichts auf: Während des Übergangs werden transiente Messwerte ausgelesen und ungefiltert an das Backend übermittelt. In einem Logistiksystem führt dies dazu, dass der erfasste Lagerbestand kurzzeitig falsche Werte annimmt und hin- und herspringt, bis sich die Waage wieder einschwingt. Eine softwareseitige Filterung – etwa durch eine Schwellwertentscheidung auf Basis der zeitlichen Änderungsrate – wäre notwendig, um stabile Messwerte zu gewährleisten, wurde im Rahmen dieser Arbeit jedoch nicht mehr umgesetzt.

5 Geräteübersicht

Abbildung 12 zeigt die beiden Prototypen nebeneinander: links die neue Waage (optimiertes Design, rosa Gehäuse) mit drei Bedientastern und verbesserter Elektronik, rechts die Ausgangslage aus der Vorgängerarbeit [4] (schwarzes Gehäuse, ohne Bedienelemente). Beide Geräte verfügen über ein E-Paper-Display zur Statusanzeige.

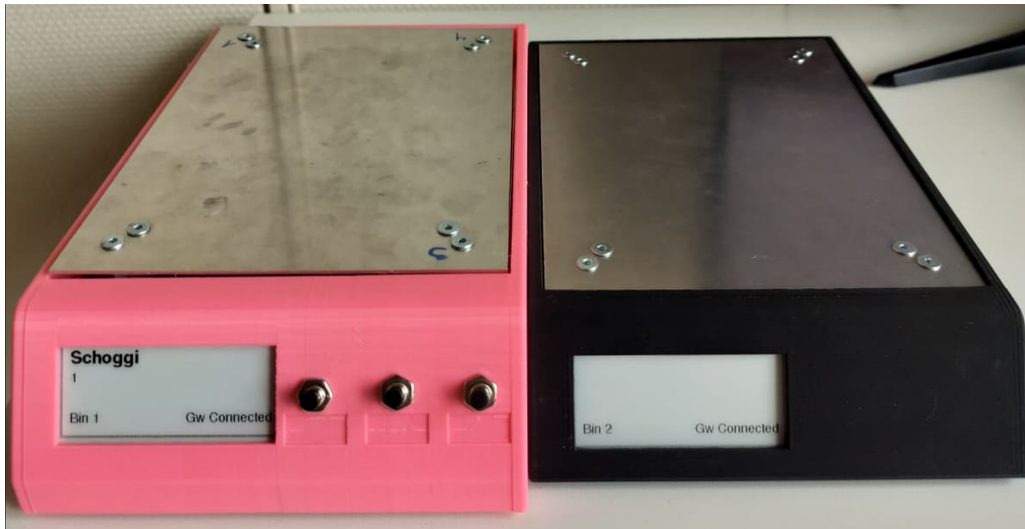


Abbildung 12: Geräteübersicht: neue Waage (links, rosa) und Vorgängerprototyp (rechts, schwarz)

6 PCB-Design

Das PCB-Layout und der vollständige Schaltplan sind im Anhang dokumentiert. Dieses Kapitel beschreibt die wesentlichen Designentscheidungen sowie die im Rahmen der Arbeit identifizierten Schwachstellen des realisierten Boards.

ADS1234 Der Einsatz des ADS1234 hat sich als richtige Entscheidung erwiesen. Dank seiner 24-Bit-Auflösung und vier unabhängigen Eingangskanälen ermöglicht er hochgenaue Einzelkanalmessungen sowie die in Abschnitt 4.2 beschriebene Ecklastkompensation.

ADS1234 – Hardware-Filterung Im PCB wurde ein analoger Filterschaltkreis (RC-Tiefpass) vor den Differenzeingängen des ADS1234 vorgesehen, jedoch nicht bestückt und nicht weiter untersucht. Die entsprechenden Bestückungsplätze wurden für den Betrieb mit Lötzinn überbrückt.

Temperatursensor SHT30 Ein SHT30-Temperatursensor wurde in das PCB integriert, um temperaturbedingte Messfehler der Wägezellen softwareseitig kompensieren zu können. Die Hardwareanbindung (I²C) ist vollständig implementiert; eine softwareseitige Auswertung wurde im Rahmen dieser Arbeit nicht realisiert.

TPS62842 Buck-Converter Für das PCB wurde der TPS62842 als Schaltregler gewählt, um die Versorgungsspannung direkt auf die für den ESP32 benötigten 3.0 V zu regeln – in der Annahme, der LM2596S könne nicht unter 5 V regeln. Dass der TPS62842 mit einer maximalen Eingangsspannung von 6.5 V für den direkten Betrieb an der Batterie (Nennspannung 7.4 V, max. 8.4 V) ungeeignet ist, war dabei bekannt und wäre in einer Folgeversion zu adressieren gewesen. Der eigentliche Irrtum wurde erst gegen Ende der Arbeit erkannt: Der LM2596S kann die Ausgangsspannung durchaus auf 3.0 V absenken – der TPS62842 wäre damit überflüssig. Die gravierendere, ebenfalls erst spät erkannte

Schwäche ist die schlechte Effizienz des LM2596S bei Leichtlast, die massgeblich für den hohen Ruhestrom im Deep Sleep verantwortlich ist (vgl. Abschnitt 3.7.2).

Messpunkte (Power-Header) Auf dem PCB wurden 2×1 -Pin-Header als Strommess-Breakouts in die Versorgungspfade der einzelnen Verbraucher eingeschleift. Diese Messpunkte haben sich bei der Leistungsanalyse als äusserst hilfreich erwiesen, da sie den direkten Anschluss des JouleScope ohne Unterbrechung der Leiterbahnen ermöglichen.

7 Messungen

7.1 Low-Power Analyse und Optimierung

7.1.1 Reduktion Versorgungsspannung 3,3 V auf 3,0 V

Da der ESP32 gemäss Datenblatt bei einer Versorgungsspannung von 3.0 V bis 3.6 V zuverlässig betrieben werden kann, wurde die Ausgangsspannung des Buck-Converters im neuen System von 3.3 V auf 3.0 V abgesenkt. Abbildung 13 und 14 zeigen den am Buck-Converter-Ausgang gemessenen Energieverbrauch für einen vollständigen Aktivzyklus – einmal mit 3.3 V und einmal mit 3.0 V Ausgangsspannung.



Abbildung 13: Messpunkt: nach Buck-Converter, Ausgang 3.3 V – neues System; Verbrauch: 416.8 mJ

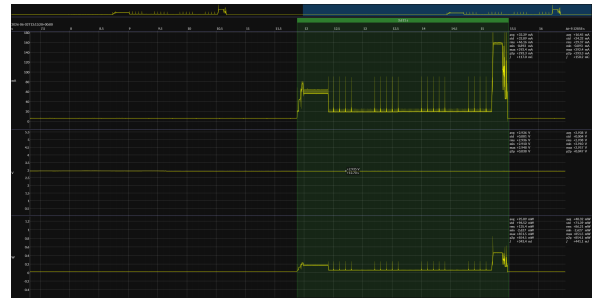


Abbildung 14: Messpunkt: nach Buck-Converter, Ausgang 3.0 V – neues System; Verbrauch: 343.4 mJ

Die Reduktion der Versorgungsspannung um 0.3 V ergibt eine Energieeinsparung von 73.4 mJ pro Zyklus, was einer Verbesserung von 17.6 % entspricht. Der Grund ist, dass der ESP32 bei niedrigerer Versorgungsspannung weniger Leistung aufnimmt.

7.1.2 Vergleich Energieverbrauch Aktiver Zyklus

ESP32 über 3V3-Pin Der Messpunkt liegt direkt am 3V3-Versorgungspin des ESP32. In der Ausgangslage wird der ESP32 mit 3.3 V gespeist, was dem Ausgang des internen Board-LDO entspricht. Im optimierten System entfällt der LDO; die Speisung erfolgt direkt über den 3V3-Pin mit 3.0 V.

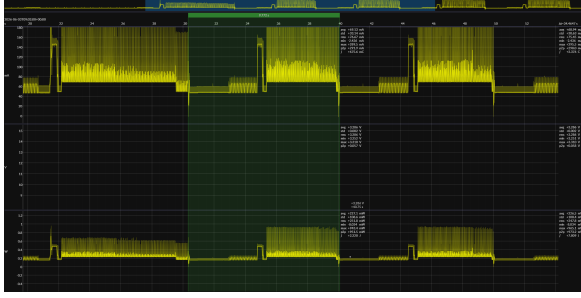


Abbildung 15: Messpunkt: vor ESP32 3V3-Pin – Ausgangslage; Wert: 2220 mJ

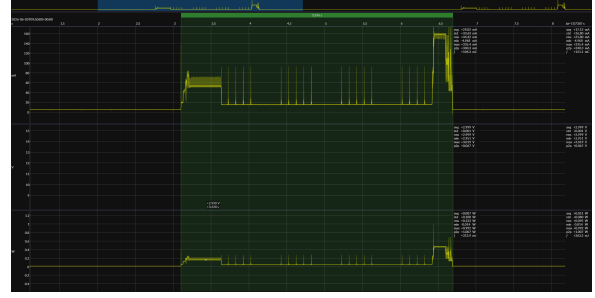


Abbildung 16: Messpunkt: vor ESP32 3V3-Pin – Optimiertes System; Wert: 312.9 mJ

Dehnmessstreifen In der **Ausgangslage** sind die Wägezellen dauerhaft mit Spannung versorgt. Der gemessene Leistungsverbrauch von 78.19 mW (Abb. 36) ergibt über die aktive Zyklusdauer von 10 s einen Energiebedarf von:

$$P_{\text{DMS}} \times t_{\text{aktiv}} = 78.19 \text{ mW} \times 10 \text{ s} = 781.9 \text{ mJ}$$

Im **optimierten System** wird der DMS-Anteil indirekt als Differenz zweier Messungen am 3V3-Pin bestimmt – einmal mit und einmal ohne eingeschaltete Dehnmessstreifen:

$$E_{\text{DMS}} = E_{\text{mit DMS}} - E_{\text{ohne DMS}} = 312.9 \text{ mJ} - 223.8 \text{ mJ} = 89.1 \text{ mJ}$$



Abbildung 17: Ganzer Zyklus **mit** DMS – Optimiertes System; Wert: 312.9 mJ

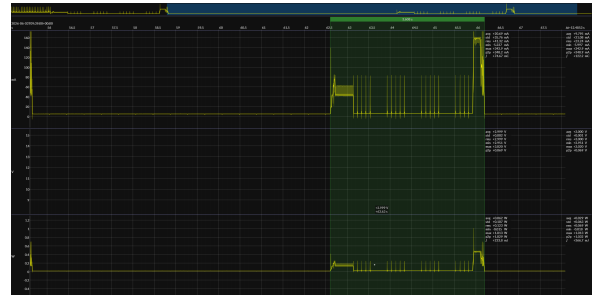


Abbildung 18: Ganzer Zyklus **ohne** DMS – Optimiertes System; Wert: 223.8 mJ

ADC Der Energiebedarf des ADC wird direkt am Versorgungspin des jeweiligen Chips gemessen.



Abbildung 19: Messpunkt: vor HX711 VIN-Pin – Ausgangslage; Wert: 106.2 mJ



Abbildung 20: Messpunkt: vor ADS1234 AVDD/DVDD-Pin – Optimiertes System; Wert: 13.42 mJ

Verluste Umwandlung Die Verluste des LDO (Ausgangslage) sowie des Buck-Converters werden in den Abschnitten 7.1.4 und 3.7.2 hergeleitet und in Tabelle 4 übernommen.

7.1.3 Vergleich Energieverbrauch ESP32

Dieser Abschnitt dokumentiert die JouleScope-Messungen am ESP32-3V3-Pin pro Betriebsphase (ohne DMS und LDO). Um Software- und Hardwareinflüsse zu trennen, wurde das alte System für diesen Vergleich ebenfalls mit 3.0 V am 3V3-Pin gespeist; die Gesamtsystemmessungen (Tabelle 4) verwenden hingegen die reale Systemkonfiguration (3.3 V im alten, 3.0 V im neuen System). Da der ADS1234 am gleichen Rail angeschlossen ist, enthält jede Phasenmessung des neuen Systems dessen Anteil. Für den fairen Vergleich wird der gemessene ADS1234-Anteil von 13.42 mJ von allen Neu-Werten abgezogen; die bereinigten Werte entsprechen Tabelle 6.

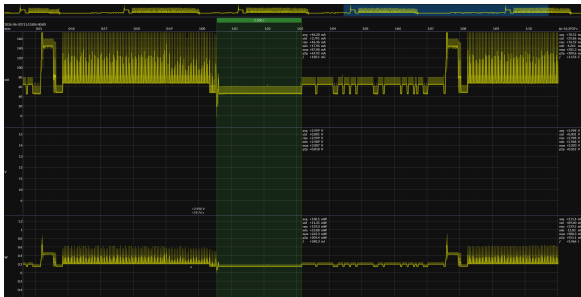


Abbildung 21: Boot – Ausgangslage; 360.3 mJ



Abbildung 22: Boot – Optimiertes System; 71.08 mJ

Boot und Ramp-up

$$E_{\text{Boot, ber.}} = 71.08 \text{ mJ} - 13.42 \text{ mJ} = 57.66 \text{ mJ}$$



Abbildung 23: ADC-Phase – Ausgangslage;
807.8 mJ



Abbildung 24: ADC-Phase – Optimiertes
System; 50.83 mJ

ADC-Messung

$$E_{\text{ADC, ber.}} = 50.83 \text{ mJ} - 13.42 \text{ mJ} = 37.41 \text{ mJ}$$



Abbildung 25: BLE-Phase – Ausgangslage;
1293 mJ



Abbildung 26: BLE-Phase – Optimiertes
System; 101.6 mJ

BLE

$$E_{\text{BLE, ber.}} = 101.6 \text{ mJ} - 13.42 \text{ mJ} = 88.18 \text{ mJ}$$

ADS1234 Chip (nur neues System) Im alten System befand sich der HX711 ausserhalb des Messpfades und wurde nicht separat erfasst. Im neuen System ist der ADS1234 am selben 3,0 V-Rail angeschlossen; sein Gesamtanteil pro Zyklus wird separat über den AVDD/DVDD-Pin gemessen und als Korrekturterm von allen Phasenmessungen abgezogen.

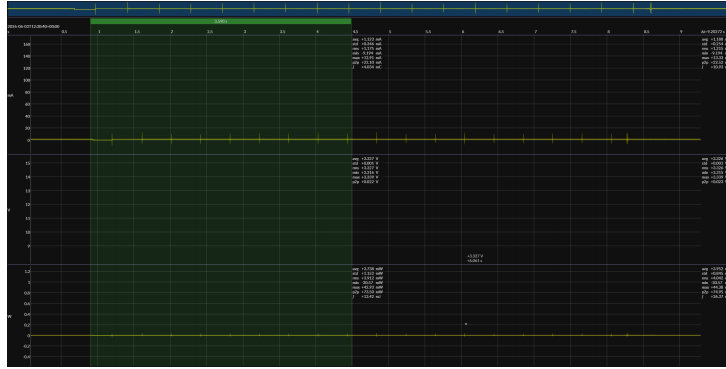


Abbildung 27: ADS1234 Chip – Optimiertes System; 13.42 mJ



Abbildung 28: Ganzer Zyklus – Ausgangslage; 1858 mJ

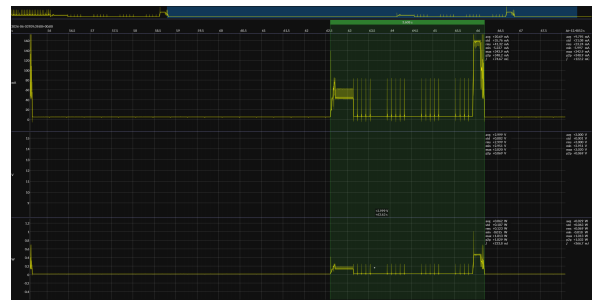


Abbildung 29: Ganzer Zyklus – Optimiertes System; 223.8 mJ

Ganzer Zyklus

$$E_{\text{Zyklus, ber.}} = 223.8 \text{ mJ} - 13.42 \text{ mJ} = 210.4 \text{ mJ}$$

7.1.4 Verluste Hardware

Buck-Converter Die Messungen erfassen jeweils die Energie *vor* (Batterieseite) und *nach* (Power-Rail) dem LM2596S-Schaltregler über einen vollständigen Zyklus. Die Herleitung des Wirkungsgrads sowie die Verlustberechnung finden sich in Abschnitt 3.7.2.

Vor dem Buck-Converter (Batterieseite):

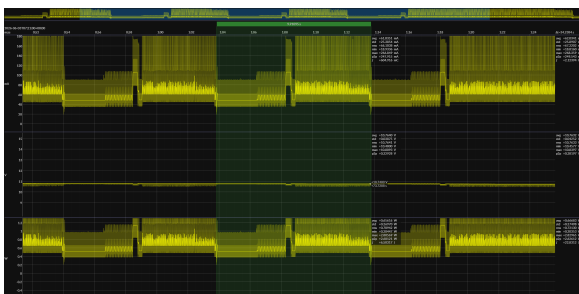


Abbildung 30: Vor Buck-Converter – Ausgangslage; 6503 mJ

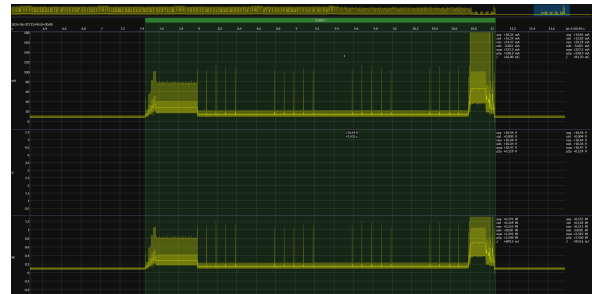


Abbildung 31: Vor Buck-Converter – Optimiertes System; 689.5 mJ

Nach dem Buck-Converter (Power-Rail):

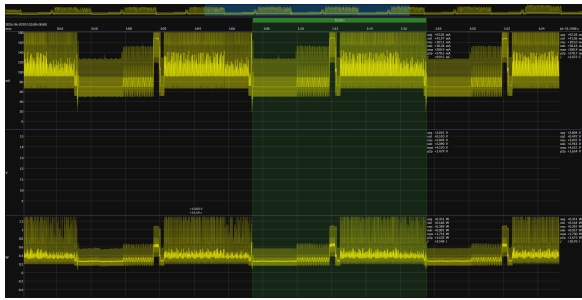


Abbildung 32: Nach Buck-Converter – Ausgangslage; 3548 mJ



Abbildung 33: Nach Buck-Converter – Optimiertes System; 343.4 mJ

LDO Der LDO ist ausschliesslich in der Ausgangslage relevant; das optimierte System speist den ESP32 direkt über den 3V3-Pin ein und umgeht den LDO vollständig.

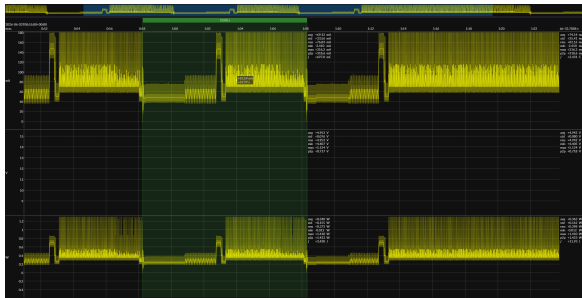


Abbildung 34: Vor LDO (VIN-Pin) – Ausgangslage; 3430 mJ

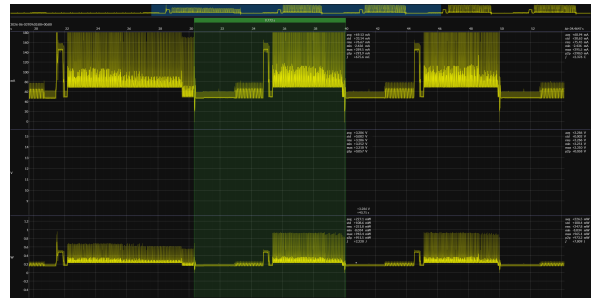


Abbildung 35: Nach LDO (3V3-Pin) – Ausgangslage; 2220 mJ

7.1.5 Vergleich Leistungsverbrauch Deep-Sleep

Dehnmessstreifen In der Ausgangslage sind die Wägezellen dauerhaft mit Spannung versorgt und verbrauchen auch im Tiefschlaf 78.19 mW. Im optimierten System werden sie vor dem Deep Sleep über den MOSFET vollständig abgeschaltet; ihr Beitrag im Tiefschlaf entfällt damit ersatzlos.

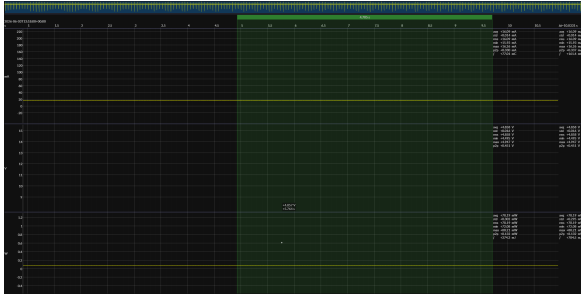


Abbildung 36: Messpunkt: vor DMS – Ausgangslage; 78.19 mW

→ Entfällt

MOSFET schaltet Versorgung
vor Deep Sleep ab.

ADC-Chip Der ADS1234 verbraucht im Tiefschlaf 9.762 mW in der Ausgangslage. Im optimierten System konnte dieser Wert auf 4.042 mW gesenkt werden. Ein vollständiges Abschalten ist jedoch nicht möglich, da kein Hardware-Pull-down am !PDWN-Pin verbaut ist und der GPIO im Deep Sleep floaten kann.



Abbildung 37: ADC-Chip im Deep Sleep – Ausgangslage (HX711); 9.762 mW

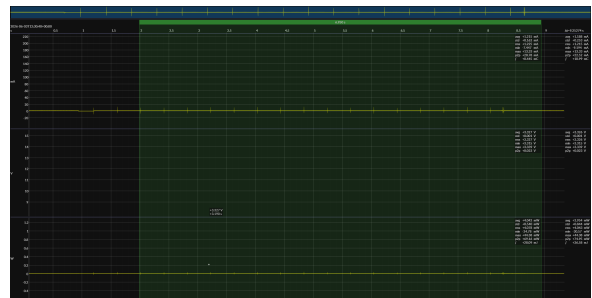


Abbildung 38: ADC-Chip im Deep Sleep – Optimiertes System (ADS1234); 4.042 mW

Vor Buck-Converter (Gesamtsystem im Deep Sleep) Die Messungen vor dem Buck-Converter erfassen den vollständigen Ruhestrom des Systems auf Batterieebene – einschliesslich aller Buck-Converter-Verluste.

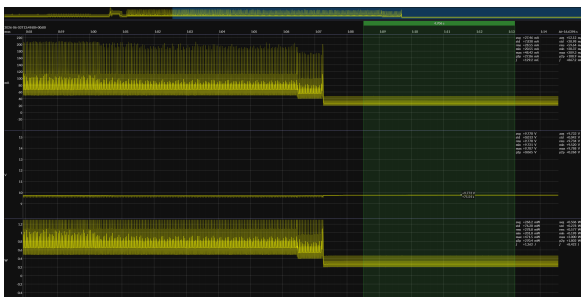


Abbildung 39: Vor Buck-Converter im Deep Sleep – Ausgangslage; 268.2 mW

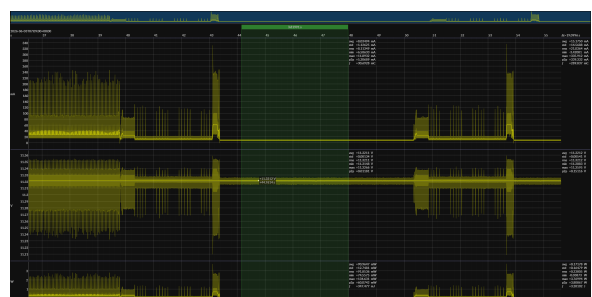


Abbildung 40: Vor Buck-Converter im Deep Sleep – Optimiertes System; 90.96 mW

Buck-Converter Leerlaufverlust Abbildung 41 zeigt den Eigenverbrauch des LM2596S ohne angeschlossene Last. Mit 69.77 mW übersteigt dieser Leerlaufverlust bereits den Gesamtverbrauch aller Verbraucher des neuen Systems (14.19 mW) um das Fünffache.

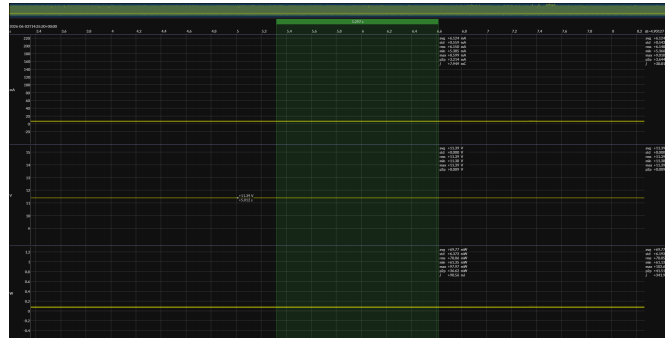


Abbildung 41: LM2596S Leerlaufverlust (kein Verbraucher angeschlossen); 69.77 mW

8 Fazit, Diskussion und Ausblick

8.1 Fazit

Diese Arbeit hatte zwei Schwerpunkte: die Optimierung des Energieverbrauchs und die Verbesserung der Messgenauigkeit einer ESP32-basierten Smart-Waage für die industrielle Inventarisierung.

Energieoptimierung. Durch den kombinierten Einsatz von Hardware- und Softwaremassnahmen konnte der Energieverbrauch pro Aktivzyklus von 6503 mJ auf 689.5 mJ gesenkt werden, was einer Reduktion von **89,4 %** entspricht. Die wirkungsvollsten Einzelmaßnahmen waren der Entfall des internen Board-LDOs durch direkte Einspeisung über den 3V3-Pin (1210 mJ gespart) sowie die Migration des BLE-Stacks von der Arduino-BLE-Bibliothek auf NimbleBLE, welche die Aktivzeit der Funkphase erheblich verkürzte. Die MOSFET-Abschaltung der Dehnmessstreifen im Tiefschlaf sowie die Reduktion der Buck-Converter-Ausgangsspannung von 3.3 V auf 3.0 V leisteten weitere messbare Beiträge. Auf Batterieebene steigt die Laufzeit gegenüber der Ausgangslage von ca. **3 Tagen** auf ca. **12 Tagen** – eine Verbesserung um den Faktor 4 (vgl. Tabelle 7).

Messgenauigkeit. Der Wechsel von einem summierten Einzelkanal auf vier individuelle ADS1234-Kanäle mit separater Kalibrierung pro Sensor ermöglicht erstmals eine Ecklastkompensation. Die Wiederholbarkeit der Messung ist mit $\pm 0,1$ g gut; die absolute Genauigkeit ist jedoch aufgrund einer unzureichenden Kalibrierung und mechanischer Mängel des Prototypen eingeschränkt – bei Mittenbelastung mit 3 kg wurden bis zu 205 g Abweichung gemessen.

8.2 Kritische Beurteilung

Trotz der erzielten Fortschritte weist das System eine Reihe fundamentaler Schwächen auf, die einer praxistauglichen Umsetzung entgegenstehen.

Entwicklungsboard und Sleep-Strom. Die Entscheidung, das SBC-NodeMCU-ESP32-C-Entwicklungsboard beizubehalten, war pragmatisch, erweist sich aber als grösste Limitierung des Systems. Das Board verbraucht im Deep Sleep ca. 13.75 mA am 3,3 V-Rail – massgeblich verursacht durch den dauernd aktiven CP2102-USB-UART-Chip, den internen LDO und die Power-LED, die auch im Tiefschlaf nicht abgeschaltet werden können. Zum Vergleich: das nRF52840 erreicht im System-OFF-Modus 0.4 μ A – einen um den Faktor **34 000** niedrigeren Ruhestrom. Die Folge ist, dass der Sleep-Strom die Energiebilanz vollständig dominiert: Die berechnete Batterielaufzeit ist nahezu unabhängig vom Messintervall (12–13 Tage bei 1 min bis 30 min Intervall), was den Nutzen längerer Schlafphasen zunichtemacht. In einem realen Logistiksystem, das auf Batterie läuft, macht dieses Board das Produkt in der vorliegenden Form **nicht industrietauglich**.

ADC-Ruhestrom durch fehlenden Pull-down. Der !PDWN-Pin des ADS1234 wird im Deep Sleep nicht durch einen Hardware-Pull-down auf LOW gehalten. Da der GPIO im unkonfigurierten Zustand floaten oder auf HIGH gezogen sein kann, bleibt der ADC möglicherweise aktiv und verbraucht unnötig ca. 1 mA im Schlaf. Dieser Fehler hätte mit einem einzigen Pull-down-Widerstand auf der Platine behoben werden können.

Ineffizienter Buck-Converter. Der eingesetzte LM2596S ist für den vorliegenden Anwendungsfall ungeeignet. Sein Leerlaufverbrauch beträgt 69.77 mW – fast fünfmal so viel wie der Gesamtverbrauch des neuen Systems im Tiefschlaf (14.19 mW). Bei der im Tiefschlaf anliegenden Teillast sinkt der Wirkungsgrad auf unter 50 %, was bedeutet, dass mehr Energie im Schaltregler verheizt als an die Last geliefert wird. Dieser Umstand wurde erst spät im Projektverlauf erkannt und konnte nicht mehr behoben werden. In Kombination mit dem hohen Board-Sleep-Strom ist der LM2596S der zweitgrösste Energiefresser des Systems. Hinzu kommt, dass keine geeignete Kombination aus effizientem Schaltregler und passendem Akku-Setup evaluiert werden konnte; die in Abschnitt 3.8 berechnete Batterielaufzeit basiert daher auf dem vorhandenen, ineffizienten LM2596S und ist nicht repräsentativ für eine optimierte Hardwarekonfiguration.

Nicht evaluierte Muss-Ziele. Zwei Muss-Ziele gemäss Projektvereinbarung wurden nicht erfüllt. Die Evaluation der Energiequelle (Batterie vs. Akku) wurde vernachlässigt; eine strukturierte Gegenüberstellung unter realistischen Lastprofilen hätte die Laufzeitberechnung auf eine solidere Grundlage gestellt. Die Evaluation des Kommunikationsprotokolls ESP-NOW wurde bewusst nicht durchgeführt: Da eine professionelle Waage langfristig auf ein energieeffizientes Custom-PCB mit einem nRF-Chip migriert werden sollte – wie im Ausblick beschrieben – und ESP-NOW auf nRF-Chips nicht verfügbar ist, wäre eine Implementierung auf dem ESP32 eine sackgassenartige Optimierung ohne Zukunft.

Messgenauigkeit. Die aktuelle Kalibrierung genügt nicht für den produktiven Einsatz. Die positionsabhängigen Abweichungen (bis –24,5 g an Ecke 2 und +35 g bei Mittenbelastung mit 1 kg) sowie die stark ausgeprägte Überschätzung bei 3 kg zeigen, dass die Kalibrierstrategie und die mechanische Ausführung des Prototypen überarbeitet werden müssen. Zudem fehlt eine Transientunterdrückung: Beim Aufbringen oder Abnehmen von Gewicht werden instabile Messwerte ungefiltert ans Backend übermittelt, was im Logistiksystem zu Bestandsschwankungen führt.

8.3 Ausblick

Die vorliegende Arbeit legt eine solide Grundlage, lässt aber klare Ansatzpunkte für folgende Arbeiten offen.

Plattformwechsel. Der wirkungsvollste einzelne Schritt wäre der Wechsel auf eine Hardware-Plattform mit minimalem Sleep-Strom. Das nRF52840 wäre mit $0.4\ \mu\text{A}$ System-OFF-Strom ideal; alternativ bietet der ESP32-C3 mit $5\ \mu\text{A}$ Deep-Sleep auf einem schlanken Custom-Board einen guten Kompromiss bei geringerem Portierungsaufwand.

Buck-Converter und Energiequelle. Der LM2596S muss durch einen modernen Schaltregler mit niedrigem Quiescent-Strom und Leichtlast-Optimierung (z. B. PFM-Modus) ersetzt werden; die konkrete Bauteilwahl muss mit dem endgültigen Akku-Setup (Zellanzahl, Spannungsbereich) abgestimmt werden. Parallel dazu sollte eine strukturierte Evaluation der Energiequelle (Lithium-Polymer-Akku vs. Primärzelle) unter realistischen Zyklusbedingungen durchgeführt werden.

ADC-Abschaltung. Ein Hardware-Pull-down-Widerstand auf dem !PDWN-Pin des ADS1234 stellt sicher, dass der ADC im Deep Sleep vollständig abgeschaltet ist und keine unnötigen $1\ \text{mA}$ verbraucht.

Messgenauigkeit und Mechanik. Eine überarbeitete Kalibrierung – idealerweise mit positionsabhängiger Gewichtung der vier Kanäle – sowie eine präzisere, formschlüssige Montage der Wägezellen im Gehäuse würden die Absolutgenauigkeit erheblich verbessern. Eine softwareseitige Transientenfilterung (z. B. Schwellwertentscheidung auf der zeitlichen Änderungsrate) sollte implementiert werden, um stabile Messwerte an das Backend zu liefern. Längerfristig wäre eine Temperaturkompensation der Dehnmessstreifen sinnvoll, da die YZC-133-Zellen temperaturbedingte Drifts aufweisen können.

Weg zum Produkt. Für eine produktionsreife Version sind ein massenproduktions-taugliches PCB-Layout, ein robustes Gehäusedesign sowie eine Ende-zu-Ende-Validierung unter realen Logistikbedingungen notwendig. Die in dieser Arbeit erzielten Software-Optimierungen und das ADS1234-Kanalkonzept stellen dabei eine wiederverwendbare Basis dar.

Literatur

- [1] ESP32 active mode and deep sleep mode power consumption comparison. <https://circuitdigest.com/microcontroller-projects/esp32-active-mode-and-deep-sleep-mode-power-consumption>. Online; aufgerufen 05.06.2026.
- [2] Kraftsensor 0...1 kg, yzc-133. https://www.reichelt.com/ch/de/shop/produkt/kraftsensor_0_1_kg_ymc-133-282443. Online; aufgerufen 29.04.2026.
- [3] Tps62842 datasheet. <https://www.ti.com/product/TPS62840/part-details/TPS62842DGRR>. Online; aufgerufen 01.06.2026.

- [4] Cyrill Berger, Dominik Müller, and Kilian Brunner. Entwicklung einer smart-waage mit offener schnittstelle für die industrie 4.0. Bachelorarbeit studiengang informatik, ZHAW, Institute of Embedded Systems (InES), June 2025. Dok.-Nr.: BA FS 25_brnn_283; Industriepartner: Skysec GmbH.
- [5] Espressif Systems. ESP32-C3 series datasheet. https://www.espressif.com/sites/default/files/documentation/esp32-c3_datasheet_en.pdf. Version 2.4; Online; aufgerufen 05.06.2026.
- [6] Joy-IT. SBC-NodeMCU-ESP32-C nodemcu esp32 usb-c – datasheet. https://joy-it.net/files/files/Produkte/SBC-NodeMCU-ESP32-C/SBC-NodeMCU-ESP32-C_Datasheet-EN_2024-12-16.pdf. Version 2024-12-16; Online; aufgerufen 05.06.2026.
- [7] Nordic Semiconductor. nRF52840 product specification. https://docs.nordicsemi.com/bundle/ps_nrf52840/page/keyfeatures_html5.html. Online; aufgerufen 05.06.2026.
- [8] Thomas Rocheteau. Vereinbarung projekt-/bachelorarbeit: Entwicklung einer smart-waage mit offener schnittstelle für die industrie 4.0. Projektvereinbarung bachelorarbeit, ZHAW, Institute of Embedded Systems (InES), February 2026. Dok.-Nr.: BA26_brnn_241; Betreuer: Kilian Brunner; Industriepartner: Skysec GmbH.
- [9] Texas Instruments. ADS1234 product datasheet. <https://www.ti.com/lit/ds/symlink/ads1234.pdf>. Online; aufgerufen 29.04.2026.
- [10] Texas Instruments. LM2596 SIMPLE SWITCHER® Power Converter 150-kHz 3-A Step-Down Voltage Regulator. <https://www.ti.com/lit/ds/symlink/lm2596.pdf>. Revision C (SNVS124C); Online; aufgerufen 01.06.2026.

A PCB Schaltplan

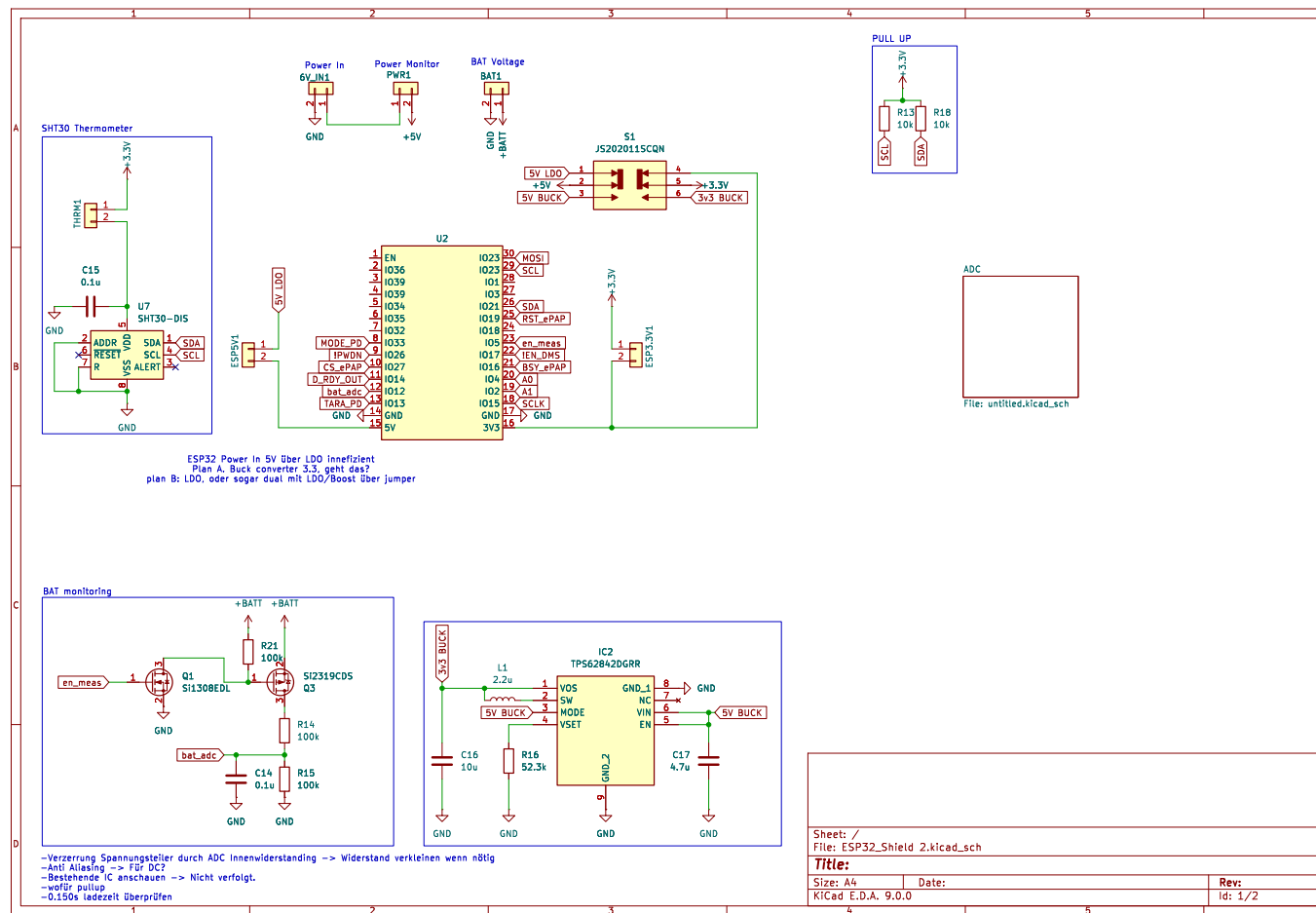


Abbildung 42: PCB Schaltplan – Seite 1

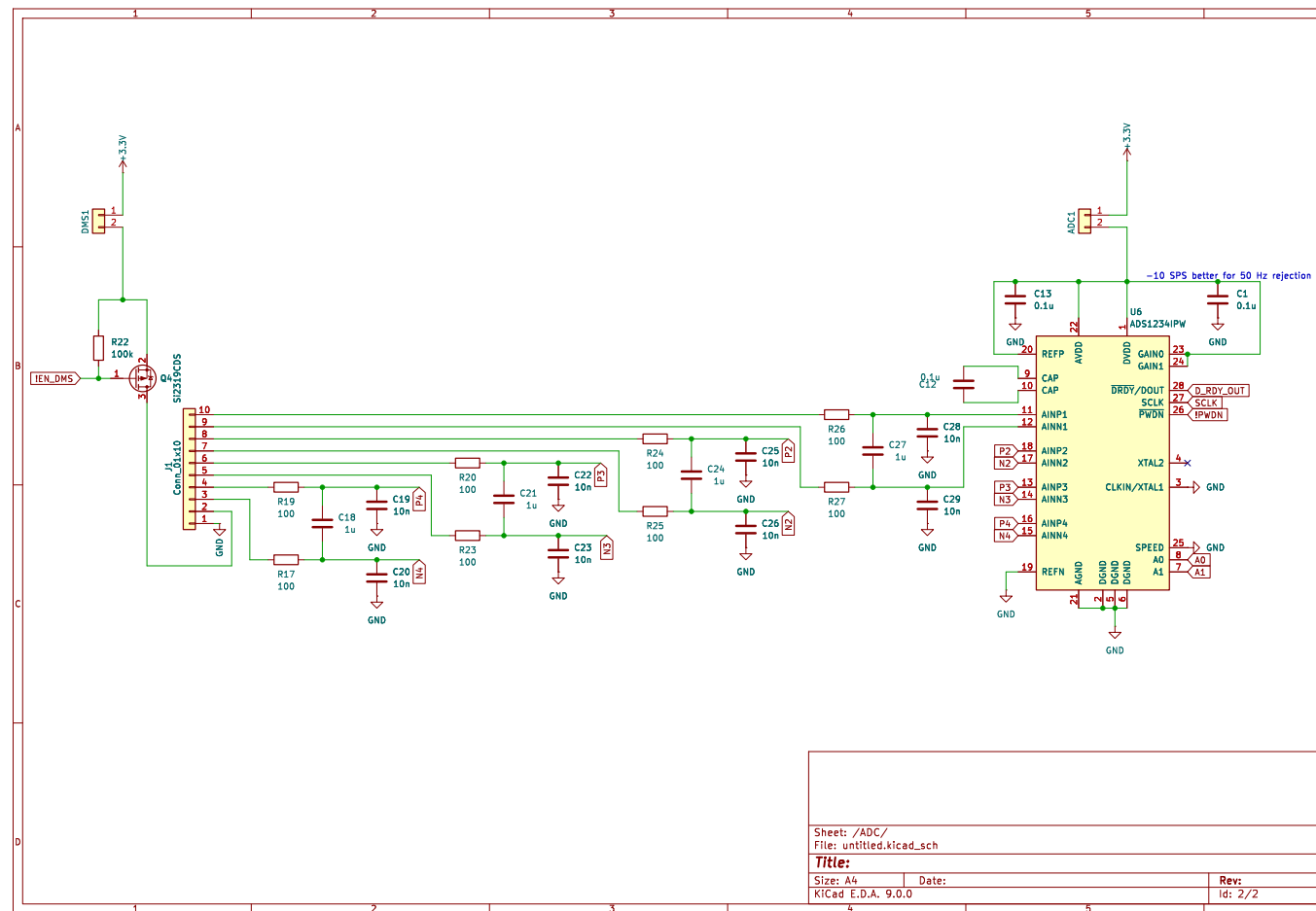


Abbildung 43: PCB Schaltplan – Seite 2

B Projektvereinbarung

Vereinbarung Projekt- / Bachelorarbeit

Identifikation: BA26_brnn_241

Entwicklung einer Smart-Waage mit offener Schnittstelle für die Industrie 4.0

Betreuer

Kilian Brunner, Dozent
ZHAW SoE; Institute of Embedded Systems

Studierende

Thomas Rocheteau

Winterthur, 16.02.2026

Version	Datum	Visum	Änderung
1.0	16.02.2026		Initiale Dokumentation

Vertraulichkeitserklärung:

Keine (wird ergänzt falls nötig)

© Copyrightinweis:

Dieses Dokument ist für den internen Gebrauch bestimmt.

Gemäss Vorgaben der ZHAW (vgl. Z-MB-Merkblatt Berechtigung an schriftlichen Studierendenarbeiten [1]) liegen die Veröffentlichungs- und Verwertungsrechte für die Arbeit selbst bei der ZHAW. Änderungen werden individuell vereinbart, was im Regelfall pragmatisch und gehandhabt wird.

Inhaltsverzeichnis

1	EINLEITUNG	3
1.1	ARBEITSTITEL DES PROJEKTS	3
1.2	FACHKONTAKTE	3
1.3	BETREUUNG	3
1.4	AUSFÜHRENDE STUDIERENDE.....	3
2	ZIELSETZUNG.....	3
2.1	AUSGANGSLAGE.....	3
2.2	AUFTRAG.....	4
2.2.1	<i>Originalausschreibungstext:</i>	<i>4</i>
2.3	ZIELSETZUNG.....	5
2.4	SYSTEMABGRENZUNG.....	5
2.4.1	<i>Randbedingungen.....</i>	<i>5</i>
2.5	RISIKEN.....	6
2.5.1	<i>Hardware-Kompatibilität der einzelnen Komponenten.....</i>	<i>6</i>
2.5.2	<i>Zeitmangel.....</i>	<i>6</i>
2.5.3	<i>Fehlendes Knowhow.....</i>	<i>6</i>
2.5.4	<i>Verzögerung bei Materialbeschaffung</i>	<i>7</i>
2.5.5	<i>Over-Engineering.....</i>	<i>7</i>
3	PROJEKTPLANUNG	7
3.1	ZEITBUDGET	7
3.2	ARBEITSPAKETE	7
3.3	MEILENSTEINE	9
3.4	GANT-DIAGRAMM ZEITPLANNUNG	10
3.5	BUDGETPLANUNG.....	11
3.5.1	<i>Allgemeine Regelungen</i>	<i>11</i>
3.5.2	<i>Kostenschätzung</i>	<i>11</i>
4	PROJEKTORGANISATION	11
4.1	PROJEKTSITZUNGEN	11
4.2	KOMMUNIKATION	11
4.3	VERTRAULICHKEIT, GEHEIMHALTUNG.....	11
5	ABGABE	11
5.1	DOKUMENTATION	11
5.2	HARD- UND SOFTWARE	11
6	BEWERTUNGSRASTER.....	12
7	UNTERSCHRIFTEN	13
8	REFERENZEN	13
8.1	VORGABEN	13

1 Einleitung

1.1 Arbeitstitel des Projekts

Entwicklung einer Smart-Waage mit offener Schnittstelle für die Industrie 4.0

1.2 Fachkontakte

Fachkontakte: Manuel Metz, Skysec (Industriepartner)

1.3 Betreuung

Hauptbetreuer: Kilian Brunner

Fachexperte (nur BA): noch offen

1.4 Ausführende Studierende

Lino Rocheteau, Thomas Klasse: Et22T

2 Zielsetzung

Dieses Kapitel beinhaltet die Zielsetzung der Bachelorarbeit zum Thema Entwicklung einer Smart-Waage mit offener Schnittstelle für die Industrie 4.0.

2.1 Ausgangslage

Ausgangslage für diese Arbeit bildet die Firma Skysec GmbH, ein Startup, welches sich auf die Herstellung von Drohnen zum Schutz von Einrichtungen und Gebieten spezialisiert hat. Für den Herstellungsprozess, insbesondere die Inventarisierung, sollen hierbei Smart-Waagen (Smart-Bins) eingesetzt werden.

Es existieren bereits Lösungen von Anbietern wie Bossard, welche jedoch auf die proprietäre Software bzw. Cloudlösung der jeweiligen Anbieter angewiesen sind. Der Industriepartner sucht deshalb nach einer Lösung mit einer offenen Schnittstelle.

Im Rahmen einer vorangegangenen Bachelorarbeit (BA FS25, Berger & Müller) wurde bereits ein Proof of Concept eines solchen Systems entwickelt. Dieses umfasst einen ESP32-basierten Waagen-Prototyp, ein Gateway auf Basis eines Raspberry Pi sowie ein containerisiertes Backend mit ASP.NET Core. Die Kommunikation zwischen Waage und Gateway erfolgt über Bluetooth Low Energy (BLE). Das System wurde erfolgreich validiert und bietet eine funktionsfähige Grundlage.

Die vorliegende Arbeit knüpft an diesen PoC an und setzt den Fokus auf die Weiterentwicklung und Optimierung der Waage selbst. Im Vordergrund stehen dabei die Evaluation alternativer Hardware-Plattformen und Kommunikationsprotokolle, die Analyse und Reduktion des Energieverbrauchs, die Entwicklung eines erweiterten Benutzerinterfaces auf der Waage sowie die Evaluation geeigneter Energiequellen für einen autonomen Betrieb.

2.2 Auftrag

2.2.1 Originalausschreibungstext:

Motivation: In der modernen Industrieproduktion steigt die Nachfrage nach intelligenten, automatisierten Lösungen für das Bestandsmanagement von Materialien und Kleinteilen. Produkte wie die sogenannten "Smart-Bins" ermöglichen es, den Nachschub von beispielsweise Schrauben in Echtzeit zu überwachen und zu steuern. Dennoch bleibt eine zentrale Herausforderung bestehen: Die meisten dieser Lösungen, etwa von Anbietern wie Bossard, sind in proprietäre Systeme eingebunden, sodass die erhobenen Daten automatisch nur in den Clouds der Hersteller landen. Dadurch wird die Integration in firmenspezifische Systeme erschwert und der Zugriff auf die Daten für eigene Analysen eingeschränkt.

Projektziel: In dieser Bachelorarbeit geht es darum, ein "offenes" Smart-Bin von Grund auf zu entwickeln, das flexibel und anpassbar in jede Produktionsumgebung integriert werden kann. Das Ziel: eine smarte Waage mit einer offenen Schnittstelle, die den direkten Zugriff auf die erfassten Bestandsdaten ermöglicht und Unternehmen die volle Kontrolle über ihre eigenen Produktionsdaten gibt.

Flexibilität und individuelle Zielsetzung: Die Studierenden haben die Freiheit, die Schwerpunkte und konkreten Ziele ihres Projekts in Absprache mit dem Betreuer selbst festzulegen. Je nach Interesse können die Studierenden entweder verstärkt hardware- oder softwareseitig arbeiten oder einen ausgewogenen Ansatz verfolgen. Nach der Annahme des Projektes sollen die Studierenden eine eigene Zielsetzung formulieren und diese zur Abstimmung mit dem Betreuer vorlegen. Dadurch wird eine individuelle Gestaltung des Projekts ermöglicht, die sowohl den persönlichen Stärken als auch den Interessen der Studierenden entspricht.

Vorgeschlagene Aufgaben und mögliche Erweiterungen:

- Grundlegende Aufgabe: Der erste Schritt in diesem Projekt ist die Auswahl geeigneter elektronischer Gewichtssensoren, die zuverlässig und präzise das Gewicht des Inhalts der Smart-Bin erfassen können. Diese Sensoren sollen dann mit einem Mikrocontroller wie dem STM32 oder Raspberry Pi verbunden werden, um die Daten auszulesen und über eine frei wählbare Schnittstelle an ein PC-System oder eine lokale Datenbank zu übertragen.
- Zusätzliche Optionen zur Vertiefung
 - Genauigkeitssteigerung: Feine Kalibrierung und Signalverarbeitung zur Verbesserung der Messgenauigkeit der Gewichtssensoren.
 - Energieeffizienz: Implementierung von Low-Power-Methoden für den Mikrocontroller, um die Lösung langlebiger und autark betreiben zu können.
 - Cloud-Integration: Eine optionale Cloud-Anbindung, die es ermöglicht, die Daten zu analysieren und in übergeordnete Systeme zu integrieren – jedoch unter der Kontrolle des Anlagenbetreibers und nicht eines externen Anbieters.

Diese Bachelorarbeit bietet damit die Möglichkeit, eine innovative, transparente und herstellerunabhängige Lösung für die Bestandsüberwachung zu entwickeln und so einen praktischen Beitrag zur Industrie 4.0 zu leisten.

2.3 Zielsetzung

Die Zielsetzung wurde nachfolgend in funktionelle und organisatorische Ziele unterteilt.

Funktionale Ziele

Muss-Ziele:

- Evaluation Hardware (ESP32, STM32, Arduino Raspberry)
- Evaluation Protokoll (Bluetooth Low Energy, ESP-NOW)
- Analyse Leistungsbedarf der Waage und Evaluation von Leistungssparmassnahmen
- Interface mit der Waage um Funktionen wie Tare, Stückzahl und Gewicht anzuzeigen, Einheitenumrechner
- Power Optimierung durch reduzieren der Aktiv- und Ruhestrome
- Evaluation Power Source (Batterie vs Akku)

Soll-Ziele:

- Temperatur Korrektur der Dehnmessstreifen

Optionale-Ziele:

- Massenproduktionstaugliches PCB
- Umportieren auf STM32
- Umbau auf Batterie

Organisatorische Ziele

Muss-Ziele:

- Erstellung eines Projektplans
- Beschaffung der benötigten Hardware
- Der Abgabetermin der Projektarbeit muss eingehalten werden.

Soll-Ziele:

- Die Meilensteine sollen eingehalten werden.

2.4 Systemabgrenzung

- On-Premise Lösung
- Evaluation des Gewichtssensors
- Dynamisches Softwareupdate (OTA)
- Einbindung in Firmennetzwerk
- Fertiges Produkt
- Verschlüsselte Übermittlung der Sensordaten (an Gateway)
- Ausarbeitung bestehendes Backend

2.4.1 Randbedingungen

- Arbeitsplätze im Raum TE514/ TE516 stehen bereit
- Messgeräte, Lötstation, Industriennetzgerät
- Der Projektstand wird regelmässig mit dem Betreuer besprochen (s. Kapitel 4)
- Sourcecode wird auf GitHub gehostet.

2.5 Risiken

Folgende Risiken wurden vor Beginn der Arbeit identifiziert. P bezeichnet die Eintretenswahrscheinlichkeit und E den Einfluss auf die Durchführung bzw. den Erfolg der Arbeit.

2.5.1 Hardware-Kompatibilität der einzelnen Komponenten

Da im Rahmen dieser Arbeit verschiedene Hardware-Plattformen evaluiert und gegebenenfalls kombiniert werden, besteht das Risiko, dass einzelne Komponenten nicht miteinander kompatibel sind. Dies könnte zu Verzögerungen im Projektverlauf führen.

P	E	Gegenmassnahmen
mittel	hoch	- Sorgfältige Recherche zur Kompatibilität der Hardware bereits in der Evaluationsphase - Rückgriff auf bewährte Komponenten aus der Vorgängerarbeit als Fallback-Option - Frühzeitiges Prototyping zur Verifikation der Kompatibilität

2.5.2 Zeitmangel

Unvorhergesehene Schwierigkeiten, insbesondere bei der Hardware-Inbetriebnahme oder der Energieverbrauchsanalyse, könnten zu einem Zeitmangel führen und sich auf die Qualität der Arbeit und Dokumentation auswirken.

P	E	Gegenmassnahmen
mittel	mittel	- Strukturierte Planung mittels Gantt-Chart - Klare Definition des Projektumfangs anhand der Zielvereinbarung - Regelmässige Überprüfung der Meilensteine - Einplanung ausreichender Zeitpuffer - Fortlaufende Pflege der Dokumentation

2.5.3 Fehlendes Knowhow

Da alternative Hardware-Plattformen wie STM32 oder neue Protokolle wie ESP-NOW evaluiert werden, könnte fehlendes Vorwissen den Projektfortschritt beeinträchtigen.

P	E	Gegenmassnahmen
mittel	niedrig	- Frühzeitige Einarbeitung in neue Plattformen und Protokolle - Nutzung der Erkenntnisse und des Codes aus der Vorgängerarbeit - Kontaktieren von Fachexperten an der ZHAW bei Bedarf

2.5.4 Verzögerung bei Materialbeschaffung

Obwohl es sich um Standard-Hardware in kleinen Mengen handelt, könnten einzelne Komponenten nicht zeitnah verfügbar sein, was zu Verzögerungen führen würde.

P	E	Gegenmassnahmen
niedrig	mittel	- Benötigte Hardware unmittelbar nach Abschluss der Evaluation bestellen - Alternative Bezugsquellen identifizieren

2.5.5 Over-Engineering

Es besteht das Risiko, dass Funktionen umgesetzt werden, die über den definierten Scope hinausgehen, oder dass eine unnötig komplexe Lösung gewählt wird.

P	E	Gegenmassnahmen
mittel	mittel	- Konsequente Orientierung an den definierten Muss-, Soll- und optionalen Zielen - Regelmässige Absprache mit dem Betreuer - Berücksichtigung des festgelegten Projektumfangs

3 Projektplanung

3.1 Zeitbudget

Die Bachelorarbeit wird im Rahmen des Frühlingssemesters 2026 durchgeführt.

Beginn der Arbeit: 10.02.2026
 Abgabe Projektdokumentation: 06.06.2026
 Verteidigung der Bachelorarbeit: 25.06.2026 – 01.07.2026

Für die Bachelorarbeit sind 12 Credits vorgesehen. Ein Credit entspricht rund 30 Arbeitsstunden. Damit betragen die geplanten Zeitressourcen für dieses Projekt 360 Arbeitsstunden.

3.2 Arbeitspakete

Eine detaillierte Zeitplanung ist separat beigelegt (Gantt-Diagramm). Die Arbeiten sind in fünf Arbeitspakete wie folgt aufgeteilt:

WP1: Projektstart, Analyse/ Planung	
Inhalt:	- Die Aufgabenstellung wird analysiert - Eine grobe Projektplanung wird erstellt
Voraussetzung:	- Projektaufgabenstellung erhalten - Zugang zu den Artefakten und dem Code der Vorgängerarbeit
Ergebnis:	- Fragen zur Aufgabenstellung geklärt

	- Unterschriebene Projektvereinbarung (dieses Dokument) - Zeitplanung (Gantt-Chart)
Ressourcen:	- Dokumentation und Artefakte der Vorgängerarbeit

WP2: Evaluation der verwendeten Hardware, Software und Protokolle/ Übertragungstechniken

Inhalt:	- Evaluation alternativer Hardware-Plattformen (ESP32, STM32, Arduino, Raspberry) - Evaluation der Kommunikationsprotokolle (BLE, ESP-NOW) - Analyse des Leistungsbedarfs der bestehenden Waage - Evaluation möglicher Energiequellen (Akku vs. Batterie) - Bestellung benötigter Hardwarekomponenten
Voraussetzung:	- WP1 abgeschlossen
Ergebnis:	- Dokumentierte Evaluationsergebnisse und Technologieentscheide - Benötigte Hardwarekomponenten bestellt
Ressourcen:	- Literatur - Hardware aus der Vorgängerarbeit

WP3: Implementierung Hardware und Software

Inhalt:	- Umsetzung der Leistungssparmassnahmen und Ruhestromoptimierung - Entwicklung des Benutzerinterfaces auf der Waage (Tare, Stückzahl, Gewicht, Einheitenumrechner) - Integration und Test der evaluierten Hardware-Komponenten
Voraussetzung:	- WP2 abgeschlossen - Bestellte Hardware erhalten
Ergebnis:	- Funktionsfähiger Prototyp mit optimiertem Energieverbrauch und erweitertem Interface
Ressourcen:	- Komponenten aus WP2 - Code der Vorgängerarbeit

WP4: Testing und Optimierung

Inhalt:	- Messungen des Energieverbrauchs und Validierung der Optimierungen - Testen des Benutzerinterfaces und der Kommunikation - Vergleichsmessungen zu Ergebnissen der Vorgängerarbeit
Voraussetzung:	- WP3 abgeschlossen

Ergebnis:	- Dokumentierte Messresultate - Getesteter und optimierter Prototyp
Ressourcen:	- Hardware und Software aus WP3

WP5: Projektabschluss und Dokumentation	
Inhalt:	- Fertigstellen der Dokumentation - Abschlusspräsentation vorbereiten
Voraussetzung:	- Praktische Arbeiten und Analysen abgeschlossen
Ergebnis:	- Dokumentation - Ggf. Poster - Vortrag
Ressourcen:	- Fertige Bachelorarbeit

3.3 Meilensteine

Nachfolgend werden alle definierten Meilensteine aufgelistet.

- 16.03.2026 Evaluation Hardware, Protokolle und Energieversorgung
- 08.05.2026 Implementierung Hardware und Software
- 29.05.2026 Testing und Optimierung
- 06.06.2026 Projektabschluss und Dokumentation

3.4 Gantt-Diagramm Zeitplanung

Nachfolgend wird das Gantt-Diagramm für den Projektverlauf gezeigt. Dieses beinhaltet alle Arbeitspakete aus Kapitel 3.2 sowie die definierten Meilensteine. Das Diagramm ist zusätzlich als PDF im Teams-Kanal im Ordner MS1 hinterlegt. Erstellt wurde es mit onlinegantt.com. Die zugehörige .gantt-Datei kann auf der genannten Webseite importiert und bearbeitet werden.



3.5 Budgetplanung

3.5.1 Allgemeine Regelungen

Beschaffungen müssen mit dem Betreuer abgesprochen und von ihm genehmigt werden. Die Beschaffung erfolgt über das InES und das Material verbleibt im Besitz des Instituts

3.5.2 Kostenschätzung

Die Hardwarekosten werden grob auf 250 CHF geschätzt. Hardware ist teilweise bereits vorhanden.

4 Projektorganisation

4.1 Projektsitzungen

Im Rahmen dieser Bachelorarbeit sind folgende Projektsitzungen vorgesehen.

- 09.02.2026: Kickoff
- Ab 19.02.2026 Weekly mit Kilian Brunner zum Projektstand

4.2 Kommunikation

Grundsätzlich erfolgt sämtliche Kommunikation per Teams oder E-mail.

4.3 Vertraulichkeit, Geheimhaltung

Aktuell besteht keine Vereinbarung. Sollte sich im Verlauf der Arbeit eine Notwendigkeit ergeben, wird dies mit dem Betreuer und dem Industriepartner abgeklärt.

5 Abgabe

5.1 Dokumentation

Die Bachelorarbeit dokumentiert das Vorgehen und die Ergebnisse in der Übersicht und im Detail. Sie folgt den Vorgaben der ZHAW *T-VL-Vorlage Berichtsstruktur PA/BA* [1]. Auf derselben Seite im Intranet finden sich auch Vorgaben zu Zusammenfassung, ein Zitierleitfaden und weitere Hilfsmittel [2].

Protokolle und Arbeitsunterlagen werden in [MS Teams](#) im entsprechenden Channel abgelegt.

5.2 Hard- und Software

Die Übergabe von Hard- und Software erfolgt nach der Präsentation der Bachelorarbeiten an den Betreuer oder eine von ihm benannte Person. Ein Exemplar des Schlussberichts wird als PDF abgegeben. Quellcode und Software in einer noch zu definierenden Form elektronisch.

Kriterium	was alles dazugehört	Gewicht
Projektverlauf, Leistung, Arbeitsverhalten	- Ingenieurmässiges Vorgehen, Methodik, Projekt- und Zeitmanagement: - Arbeitsweise Anfangsphase: Problemanalyse, Recherche, Untersuchung von Lösungsvarianten - Arbeitsweise Endphase: Umgang mit Schwierigkeiten, Austesten von Varianten - Kommunikation mit Betreuer / Industriepartner, Informationsaustausch, Selbständigkeit - Organisation der Gruppe, Aufgabenteilung, Zusammenarbeit - Engagement: Initiative, Einsatz, Arbeitsleistung Kommentar: ...	33.3% (25 – 50%)
Qualität der Ergebnisse	- Niveau der Arbeit - Qualität und Vollständigkeit der Resultate - vereinbarte Ziele erreicht - signifikanter Zusatznutzen der Lösung erreicht - Verständliches Konzept, sinnvoller Einsatz von Methoden, Qualität der Tests - Originalität, Ideenreichtum, Kreativität Kommentar: ...	33.3% (25 – 50%)
Form und Inhalt des Berichts und der Präsentation	Bericht - Resultate verständlich und nachvollziehbar dargestellt - Resultate kritisch diskutiert, Verbesserungsvorschläge - Wissenschaftlichkeit (Aufbau, Referenzen, fundiert) - Formale Korrektheit (Zusammenfassung, engl. Abstract, Einleitung, Grundlagen, Pflichtenheft, Schlusswort, Quellcode, Blockschaltbild, Stückliste, Konfigurationsdaten, Installationsanleitung, Bedienanleitung, Testdokumentation) - Erscheinung, Übersichtlichkeit, Sprache - Arbeitsteilung im Team sichtbar Präsentation und Diskussion - Auftreten, Verständlichkeit, Einführung in das Thema - übersichtliche Gliederung, Beschränkung auf Wesentliches - Antworten auf Fragen, kritische Diskussion Kommentar: ...	33.3% (25 – 50%) Total 100%

7 Unterschriften

Mit der Unterschrift erklären sich die involvierten Personen mit der Projektklärung einverstanden.

Ort, Datum:

Betreuer:



Kilian Brunner

Betreuer

Ort, Datum:

Studierende:



Thomas Rocheteau

8 Referenzen

8.1 Vorgaben

- [1] Intranet der School of Engineering:

<https://intra.zhaw.ch/departemente/school-of-engineering/bachelorstudium/projekt-und-bachelorarbeiten/>

- [2] Vorlagen Titelblatt etc. im Intranet:

<https://intra.zhaw.ch/departemente/school-of-engineering/bachelorstudium/projekt-und-bachelorarbeiten/vorlagen-paba/>

C Quellcode

Der vollständige Quellcode dieser Arbeit ist öffentlich auf GitHub verfügbar:

<https://github.com/Newtonsohn/BA-Smart-Waage>

Das Repository enthält die Firmware des ESP32 (PlatformIO/Arduino-Framework), die Mess- und Auswertungsskripte sowie die LaTeX-Quelldateien dieser Dokumentation.